计 算 机 体 系 结 构

第五讲

计算机科学与技术学院
舒燕君

Recap

- 程序的局部性原理
- CPU性能公式
- 指令系统概述
  - ✓ 指令的表示方法
  - ✓ 指令系统与计算机的性能
- 指令系统的分类
  - ✓堆栈、累加器、寄存器
  - ✓ 通用寄存器型指令集结构分类及优缺点
- 第一章作业和第二章作业一起交

1.3.3 量化设计的基本原则

4. CPU性能公式

- 执行一个程序所需的CPU时间

  CPU时间 = 执行程序所需的时钟周期数×时钟周期时间

  其中：时钟周期时间是系统时钟频率的倒数。

- 每条指令执行的平均时钟周期数CPI

  CPI = 执行程序所需的时钟周期数 / IC

  IC：所执行的指令条数

- 程序执行的CPU时间可以写成

  CPU时间 = IC ×CPI ×时钟周期时间

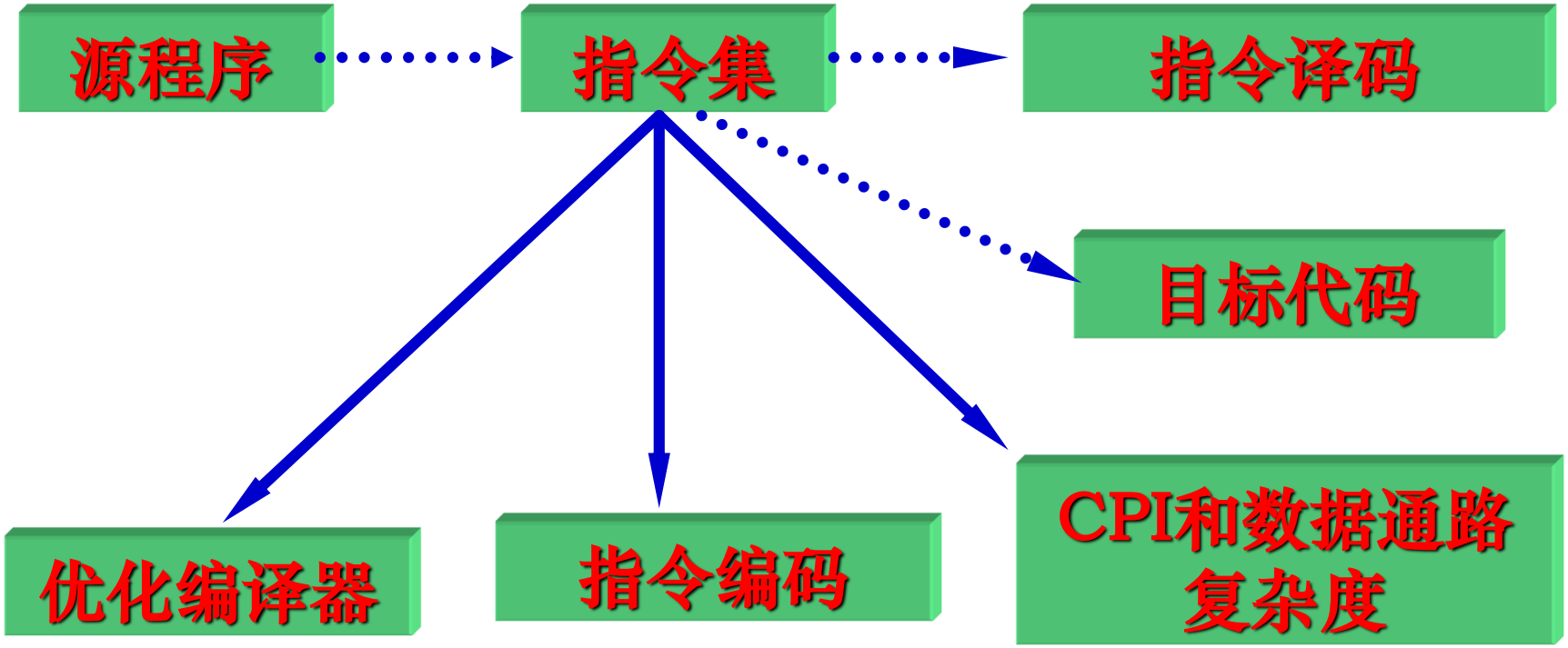
  时钟周期时间：取决于硬件实现技术和计算机组成

  CPI：取决于计算机组成和指令系统的结构；

  IC：取决于指令系统的结构和编译技术

指令系统与计算机的性能

$$T_{CPU} = CPI \times IC \times T_{CLK}$$



2.2 指令集结构的分类

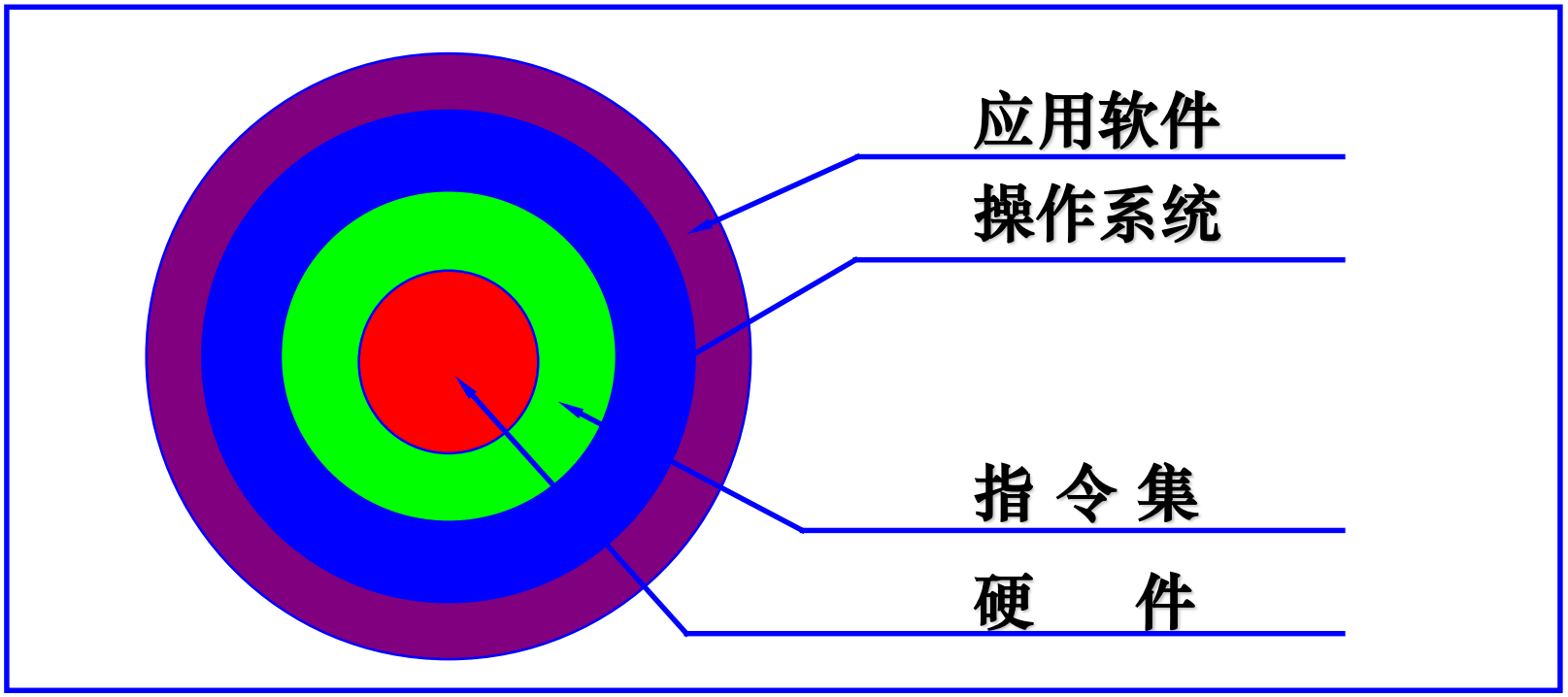
- Z=X+Y表达式在这三种类型指令集结构上的实现方法

堆栈	累加器	寄存器 (寄存器-存储器)	寄存器 (寄存器-寄存器)
PUSH X PUSH Y ADD POP Z	LOAD X ADD Y Store Z	LOAD R1,X ADD R1,Y Store R1,Z	LOAD R1,X LOAD R2,Y ADD R3,R1,R2 Store R3,Z

三种类型指令集结构的优缺点

指令集结构类型	优点	缺点
寄存器-寄存器（0，3）	指令字长固定，指令结构简洁，是一种简单的代码生成模型，各种指令的执行时钟周期数相近	与指令中含存储器操作数的指令系统结构相比，指令条数多，目标代码不够紧凑，因而程序占用的空间比较大
寄存器-存储器型（1，2）	可以在ALU指令中直接对存储器操作数进行引用，而不必先用load指令进行加载，容易对指令进行编码，目标代码比较紧凑	由于有一个操作数的内容将被破坏，所以指令中的两个操作数不对称。在一条指令中同时对寄存器操作数和存储器操作数进行编码，有可能限制指令所能够表示的寄存器个数。指令的执行时钟周期因操作数的来源（寄存器或存储器）的不同而差别比较大
存储器-存储器（2，2）或（3，3）	目标代码最紧凑，不需要设置存储器来保存变量	指令字长变换很大，特别是3个操作数指令。而且每条指令完成的工作也差别很大。对存储器的频率访问会使存储器成为瓶颈。这种类型的指令系统现在已经不用了

指令集结构设计概观



操作码	寻址方式	操作数	寻址方式	操作数

第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

四大主流ISA

序号	架构	特点	代表性厂商	运营机构	发明时间
1	x86	性能高、速度快、兼容性好、实用性强	Intel、AMD	Intel	1978
2	ARM	成本低、低功耗	高通、Nivda、Pythium、Huawei	ARM	1983
3	RISC-V	完全开源、架构简单、模块化、极简、可扩展	Samsung、阿里	RISC-V	2014
4	MIPS	简洁、优化方便包含大量寄存器、高拓展性	Loongson、MIPS	MIPS	1981

Intel 8086

(1) 指令字长　1 ~ 6 个字节

INC　AX　1 字节

MOV　WORD PTR[0204], 0138H　6 字节

(2) 地址格式

零地址	NOP		1 字节	
一地址	CALL	段间调用	5 字节	
	CALL	段内调用	3 字节	
二地址	ADD AX，BX		2 字节	寄存器 – 寄存器
	ADD AX，3048H		3 字节	寄存器 – 立即数
	ADD AX，[3048H]		4 字节	寄存器 – 存储器

ARM

1 基本格式 <opcode>{<cond>}{S}<Rd><Rn><operand2>

<> 为必选项，{}为可选项
{} 决定指令执行条件域
{S} 决定指令执行是否影响当前程序状态寄存器CPSR的值

2 指令字长32位为例

cond	00	X	opcode	S	Rn	R_d	Shifter-operand
4	2	1	4	2	4	4	12

opcode：指令操作码
cond：指令执行条件
S：指令的操作是否影响CPSR的值
Rn：第一个操作数的寄存器地址
Rd：目标寄存器地址
Shifter_operand:第二个操作数

RISC-V

1 指令集组成

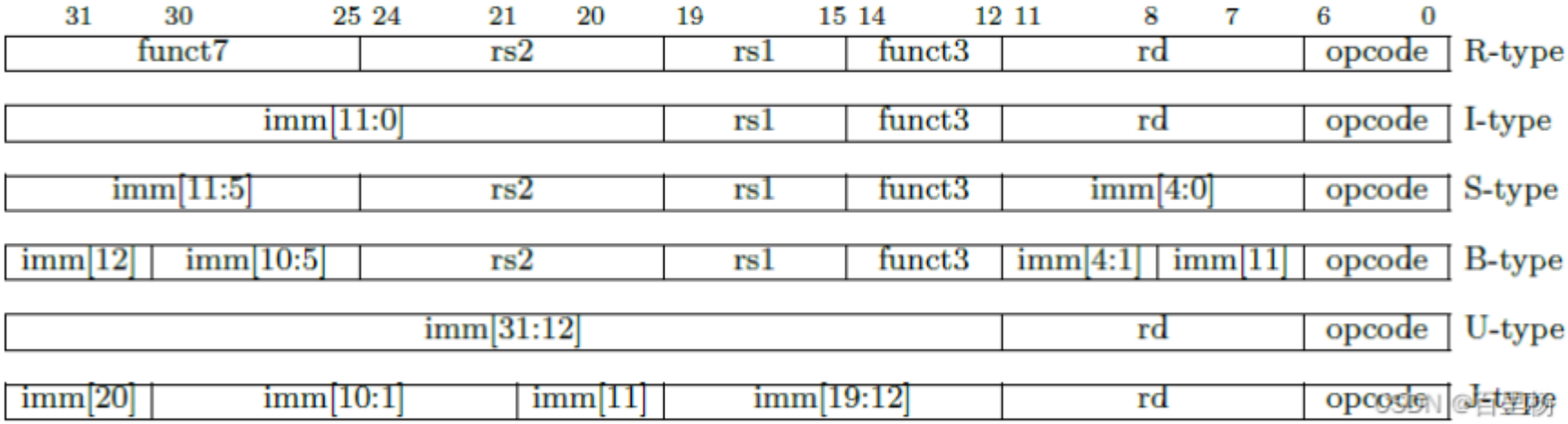
一个基础整数指令集为核心，根据需要再选择多个扩展指令集

G = I + M + A + D + F

I：基础整数指令集，M：乘法和除法指令，A：原子指令

F：当精度浮点，D：双精度浮点

2 指令字长32位为例



R型：寄存器间操作，I型：短立即数和取数，S型：存数
B型：条件分支跳转，U型：长立即数，J型：无条件跳转

2.7 MIPS指令分析

介绍MIPS32的一个子集，简称为MIPS。

- Load/Store型指令集结构
- MIPS是一种多元指令集结构
  - 体现了当今多种机器（AMD29K、DEC station 3100、HP850、IBM 801、MIPS M/120A、MIPS M/1000、Motorola 88k、RISC I、SGI4D/60、SPARC station 1、Sun 4/110、Sun 4/260等）的指令集结构的共同特点。
  - 还将会体现未来一些机器的指令集结构的特点。

MIPS指令集结构

- 具有一个简单的Load/Store指令集；
- 注重指令流水效率；
- 简化指令的译码；
- 高效支持编译器。

MIPS指令集结构：寄存器

R0	R1	R2	R3
R4	R5	R6	R7
R8	R9	R10	R11
R12	R13	R14	R15
R16	R17	R18	R19
R20	R21	R22	R23
R24	R25	R26	R27
R28	R29	R30	R31

32个32位的通用寄存器（GPRs）。

寄存器R0的内容恒为全0。

MIPS指令集结构：寄存器

F0	F1	F2	F3
F4	F5	F6	F7
F8	F9	F10	F11
F12	F13	F14	F15
F16	F17	F18	F19
F20	F21	F22	F23
F24	F25	F26	F27
F28	F29	F30	F31

32个32位浮点寄存器（FPRs）。

单精度浮点数表示和双精度浮点数表示。

MIPS指令集结构：数据类型

- 整型数据：
  - 8位、16位、32位。
- 浮点数据：
  - 32位单精度浮点；
  - 64位双精度浮点；
  - IEEE 754标准。

00000000	00000000	00000000	01100100
11111111	11111111	10011000	01100100

MIPS指令集结构：寻址方式

- 寄存器寻址；如 **ADD R1, R2, R3**
- 立即值寻址；如 **ADD R1, R2, #42**
- 偏移寻址；　如 **LW R2, 40(R3)**
  - 寄存器间接寻址；　如 **LW R2, 0(R3)**
  - 直接寻址；如 **LW R2, 40(R0)**

MIPS指令集结构：指令格式

I 类型指令

6	5	5	16
操作码	rs1	rd	立即值

字节、半字、字的载入和存储；

rd ←rs1 op 立即值。

MIPS指令集结构：指令格式

R 类型指令

6	5	5	5	11
操作码	rs1	rs2	rd	Func

寄存器－寄存器 ALU 操作：rd←rs1 func rs2；
函数对数据的操作进行编码：加、减、…；
对特殊寄存器的读/写和移动。

MIPS指令集结构：指令格式

J 类型指令

6	26
操作码	与 PC 相加的偏移量

跳转，跳转并链接，从异常（exception）处自陷和返回。

MIPS指令集结构：操作类型

- **Load和Store操作；**
- **ALU操作；**
- 分支和跳转操作；
- 浮点操作。

MIPS指令集结构：操作类型

- 符号“←”表示数据传送操作，其后附带一个下标n，也即“←n” 表示传送一个n位数据。
- 符号“##”用来表示两个域的串联操作，它可以出现在数据传送操作的任何一边。

MIPS指令集结构：操作类型

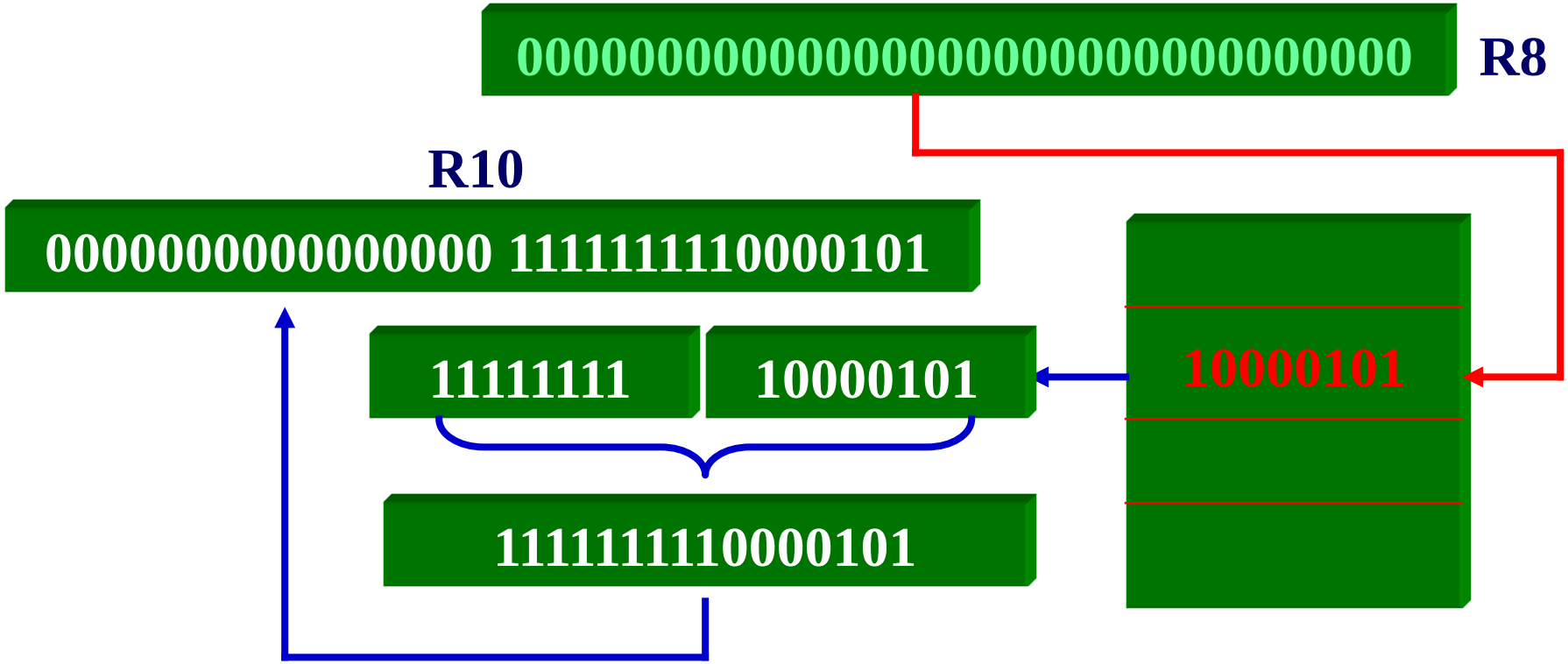
- 域的下标用来表明从该域中选择某一位。域中位的标记是从最高位开始标记，并且起始标记为0。下标可以是一个单独的数字，如$Regs[R4]_0$表示选择寄存器R4中内容的符号位；下标也可以是一个范围，如$Regs[R3]_{24..31}$表示选择寄存器R3中内容的最低一个字节。

MIPS指令集结构：操作类型

- 上标表示复制一个域，如0^{24}可以得到一个24位全为0的一个域。
- 变量Mem用来表示存储器中的一个数组，存储器按照字节寻址，它可以传送任何数目的字节。

MIPS指令集结构：操作类型

$$Regs[R10]_{16..31} \leftarrow_{16} (Mem[Regs[R8]]_0)^8 \,\#\#\, Mem[Regs[R8]]$$



MIPS指令集结构：Load和Store

- **Load和Store操作**：可以对MIPS的所有通用寄存器和浮点寄存器进行Load（载入）和Store（储存）操作，**但是对通用寄存器R0的Load操作没有任何效果**。

指令举例	指令名称	含 义
LW R2，40(R3)	装入字	Regs[R2] $\leftarrow_{32}$ Mem[40+Regs[R3]]
LB R2，30(R3)	装入字节	Regs[R2] $\leftarrow_{32}$ $(\text{Mem}[30+\text{Regs}[R3]]_0)^{24}$ ## Mem[30+Regs[R3]]
LBU R2，40(R3)	装入无符号字节	Regs[R2] $\leftarrow_{32}$ 0^{24} ## Mem[40+Regs[R3]]
LH R2，30(R3)	装入半字	Regs[R2] $\leftarrow_{32}$ $(\text{Mem}[30+\text{Regs}[R3]]_0)^{16}$ ## Mem[30+Regs[R3]]## Mem[31+Regs[R3]]
LS F2，60(R4)	装入单精度浮点	Regs[F2] $\leftarrow_{32}$ Mem[60+Regs[R4]]
SW 300(R5)，R4	保存字	Mem[300+Regs[R5]] $\leftarrow_{32}$ Regs[R4]
SH 502(R4)，R5	保存半字	Mem[502+Regs[R4]] $\leftarrow_{16}$ $\text{Regs}[R5]_{16..31}$
SB 41(R3)，R2	保存字节	Mem[41+Regs[R3]] $\leftarrow_{8}$ $\text{Regs}[R2]_{24..31}$
SS 40(R2)，F2	保存单精度浮点数	Mem[40+Regs[R2]] $\leftarrow_{32}$ Regs[F2]

MIPS指令集结构：ALU操作

- **ALU操作**：在MIPS中，所有的ALU指令都是寄存器—寄存器型指令，其运算包含了简单的算术和逻辑运算，如加、减、AND、OR、XOR和移位。
- “**设置相等**”、“**设置不等**”、“**设置小于**”：寄存器比较指令（=，≠，<，>，≤，≥），如果比较结果为真，这些指令就在目标寄存器中填入1（表示真），否则填入0（表示假）。

MIPS指令集结构：ALU操作

指令实例	指令名称	含 义
ADD R1,R2,R3	加	Regs[R1] ← Regs[R2] + Regs[R3]
ADDI R1,R2,#3	和立即值相加	Regs[R1] ← Regs[R2] + 3
LHI R1,#42	载入高位立即值	Regs[R1] ← 42 ## 0^{16}
SLLI R1,R2,#5	逻辑左移立即值形式	Regs[R1] ← Regs[R2] <<5
SLT R1,R2,R3	设置小于	if (Regs[R2] <Regs[R3]) Regs[R1] ← 1 else Regs[R1] ← 0

MIPS指令集结构：转移操作

- 描述目标地址的方法：
  - 用带符号位的26位偏移量加上程序计数器的值来确定跳转的目标地址；
  - 指定一个寄存器，由寄存器中的内容决定跳转的目标地址。

MIPS指令集结构：转移操作

- 两种跳转类型：
  - 一种是简单跳转；
  - 另一种是跳转并链接（用于过程调用），它将下一条顺序指令地址（返回地址）保存在寄存器R31中。

MIPS指令集结构：转移操作

指令实例	指令名称	含义
J name	跳转	PC ←name+PC+4; -2^{25}≤name ≤2^{25}
JAL name	跳转并链接	Regs[R31] ← PC+4; PC ←name +PC+4;-2^{25}≤name ≤2^{25}
JR R3	寄存器型跳转	PC ← Regs[R3];
JALR R2	寄存器型跳转并链接	Regs[R31]←PC+4; PC← Regs[R2];
BEQZ R4 , name	“等于0”分支	if (Regs[R4]==0) PC ← name+ PC+4;-2^{15}≤name ≤2^{15}
BNQZ R4 , name	“不等于0”分支	if (Regs[R4]!=0) PC ← name+ PC+4;-2^{15}≤name ≤2^{15}

MIPS指令集结构：浮点操作

- 浮点操作：浮点指令的操作数来源于浮点寄存器，同时它还指明了相应的操作是单精度浮点操作还是双精度浮点操作。
  - 后缀D代表双精度浮点操作
  - 而后缀S代表单精度浮点操作
  - 如：ADDD、ADDS、SUBD、SUBS、MULTD、MULTS、DIVD、DIVS。
- MIPS的浮点操作有：加、减、乘、除。

第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

2.8 MIPS的一种简单实现

- 处理器的指令字长为32位，包含
  - 32个32位通用寄存器R0~R31（R0值为0）；
  - 1个32位的指令寄存器IR；
  - 1个32位的程序计数器PC

- 取指令时，可以直接从指令存储器中提取32位的指令信息。取数据时，与数据存储器进行32位的数据交换。处理器的地址总线宽度是32位，数据总线宽度也是32位，无论是取指还是数据访问，都假设能在一个时钟周期内完成。

- 只讨论整数指令的实现（包括：Load和Store，等于0转移，整数ALU指令等。）

处理器功能及指令系统定义

- 存数指令 SW 30(R2), R1
- 取数指令 LW R1, 30(R2)

- 逻辑与指令 AND R1, R2, R3
- 逻辑或指令 OR R1, R2, R3

- 立即数加法指令 ADDI R4, R5, #6
- 立即数减法指令 SUBI R4, R5, #6

- 条件转移（零则转）指令 BEQZ R3, x
- 无条件转移指令 J x

给定的指令系统

J x	PC ← PC+x
BEQZ R3, x	If(R3=0) then PC ← PC+x
AND R1, R2, R3	Regs[R1] ←Regs[R2] and Regs[R3]
OR R1, R2, R3	Regs[R1] ←Regs[R2] or Regs[R3]
ADDI R4, R5, #6	Regs[R4] ←Regs[R5] + 6
SUBI R4, R5, #6	Regs[R4] ←Regs[R5] - 6
LW R1, 30(R2)	Regs[R1] ← Mem[30+Regs[R2]]
SW 30(R2), R1	Mem[30+Regs[R2]]← Regs[R1]

操作码

操作码占据了6位，最多可支持64种指令的设计。目前的指令系统仅包含了8种操作，下表定义这8种操作的操作码

指令名称	助记符	二进制操作码
无条件跳转	J	000000
条件跳转	BEQZ	000010
逻辑与操作	AND	000100
逻辑或操作	OR	000100
立即数减法	SUBI	001000
立即数加法	ADDI	001010
存数操作	SW	001100
取数操作	LW	001110

每条指令的格式描述

J name

0–5	6–31
000000	x

PC←PC+x

BEQZ R3,X

0–5	6–10	11–15	16–31
000010	R3		x

if R3=0 then PC←PC+x

AND R1,R2,R3

0–5	6–10	11–15	16–20	21–31
000100	R2	R3	R1	00...001

R1←R2 and R3

OR R1,R2,R3

0–5	6–10	11–15	16–20	21–31
000100	R2	R3	R1	00...010

R1←R2 or R3

SUBI R4,R5,#6

0–5	6–10	11–15	16–31
001000	R5	R4	00...0110

R4←R5-6

ADDI R4,R5,#6

0–5	6–10	11–15	16–31
001010	R5	R4	00...0110

R4←R5+6

SW 30(R2),R1

0–5	6–10	11–15	16–31
001100	R2	R1	00...00110000

Mem[R2+30]←R1

LW R1,30(R2)

0–5	6–10	11–15	16–31
001110	R2	R1	00...00110000

R1←Mem[R2+30]

指令格式举例

- 38410030
  00111000010000010000000000110000
  LW R1,30(R2)

- 10430801
  00010000010000110000100000000001
  AND R1,R2,R3

2.8 MIPS的一种简单实现

实现MIPS指令的一种简单数据通路

- 将指令执行划分为5个阶段
  - 取指令周期
  - 指令译码/读寄存器周期
  - 执行/有效地址计算周期
  - 存储器访问/分支完成周期
  - 写回周期

2.8 MIPS的一种简单实现

1.取指令周期(IF)

操作为：

根据PC值从存储器中取出指令，并将指令送入指令寄存器IR；PC值增加4，指向顺序的下一条指令，并将下一条指令的地址放入临时寄存器NPC中。

$$IR \leftarrow IMem[PC]$$

$$NPC \leftarrow PC+4$$

图示

2.8 MIPS的一种简单实现

2.指令译码/读寄存器周期(ID)

操作为：

进行指令译码，读IR寄存器（指令寄存器），按照相应寄存器号读寄存器文件，并将读出结果放入两个临时寄存器A和B中。同时对IR寄存器中内容的低16位进行符号扩展，然后将符号扩展之后的32位立即值保存在临时寄存器Imm中。

$$A \leftarrow Regs[IR_{6..10}]$$

$$B \leftarrow Regs[IR_{11..15}]$$

$$Imm \leftarrow ((IR_{16})^{16} \#\# IR_{16..31})$$

MIPS的固定字段译码技术:图示

2.8 MIPS的一种简单实现

3.执行/有效地址计算周期(EX)

操作为：

存储器访问：	**ALUoutput←A+Imm**
寄存器-寄存器ALU：	**ALUoutput←A func B**
寄存器-立即值ALU:	**ALUoutput←A op Imm**
分支操作：	**ALUoutput←NPC+Imm**
	Cond←(A op 0)

问题：为什么执行和有效地址计算可以合并？

2.8 MIPS的一种简单实现

4.访存/分支操作完成周期(MEM)

操作为：

访存操作：

Load： **LMD←DMem[ALUoutput]**

Store： **DMem[ALUoutput]←B**

分支操作： **if (Cond) PC←ALUoutput**

else PC←NPC

2.8 MIPS的一种简单实现

5.写回周期(WB)

操作为：

- 寄存器-寄存器型ALU指令：

  $$Reg[IR_{16..20}] \leftarrow ALUoutput$$

- 寄存器-立即值型ALU指令：

  $$Reg[IR_{11..15}] \leftarrow ALUoutput$$

- Load指令：

  $$Reg[IR_{11..15}] \leftarrow LMD$$

2.8 MIPS的一种简单实现

6.性能分析

在该数据通路上，

分支指令需要4个时钟周期

其它指令需要5个时钟周期

假设分支指令占总指令数的12%，问CPI=?

$$CPI=4\times12\%+5\times(1-12\%)=4.88$$

结论：就性能和硬件开销而言，上述实现不是一种优化实现！

2.8 MIPS的一种简单实现

7.改进方法

(1)在Mem周期完成ALU指令

假设ALU指令数占指令总数的44%，则在时钟周期时间不变的同时，CPI可以降低至4.44

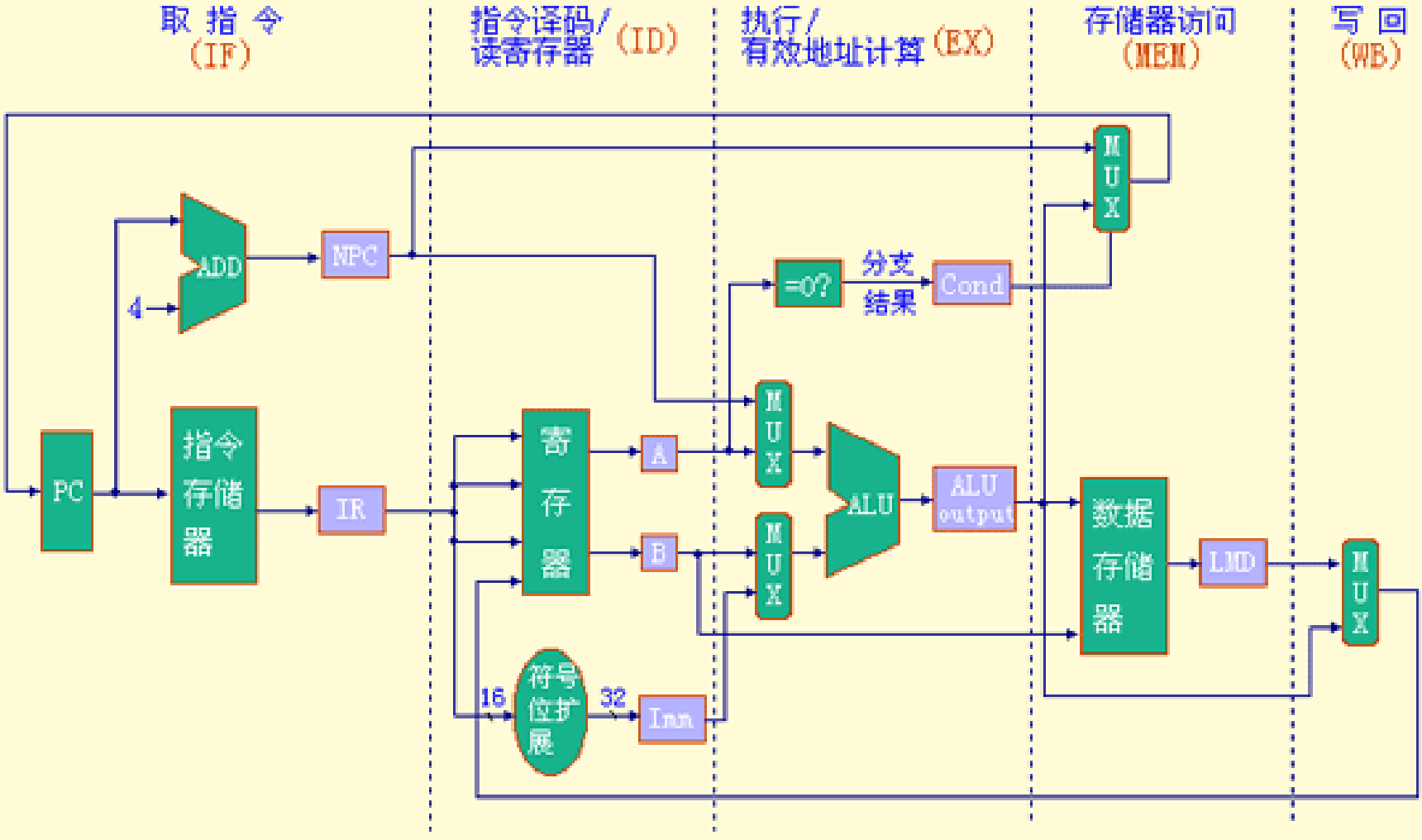
(2)如要进一步降低CPI，可能需要延长时钟周期时间，使每个时钟周期中能够完成更多的工作

(3)采用单周期实现，可以将CPI降低为1，但时钟周期时间却会增加为原来的5倍

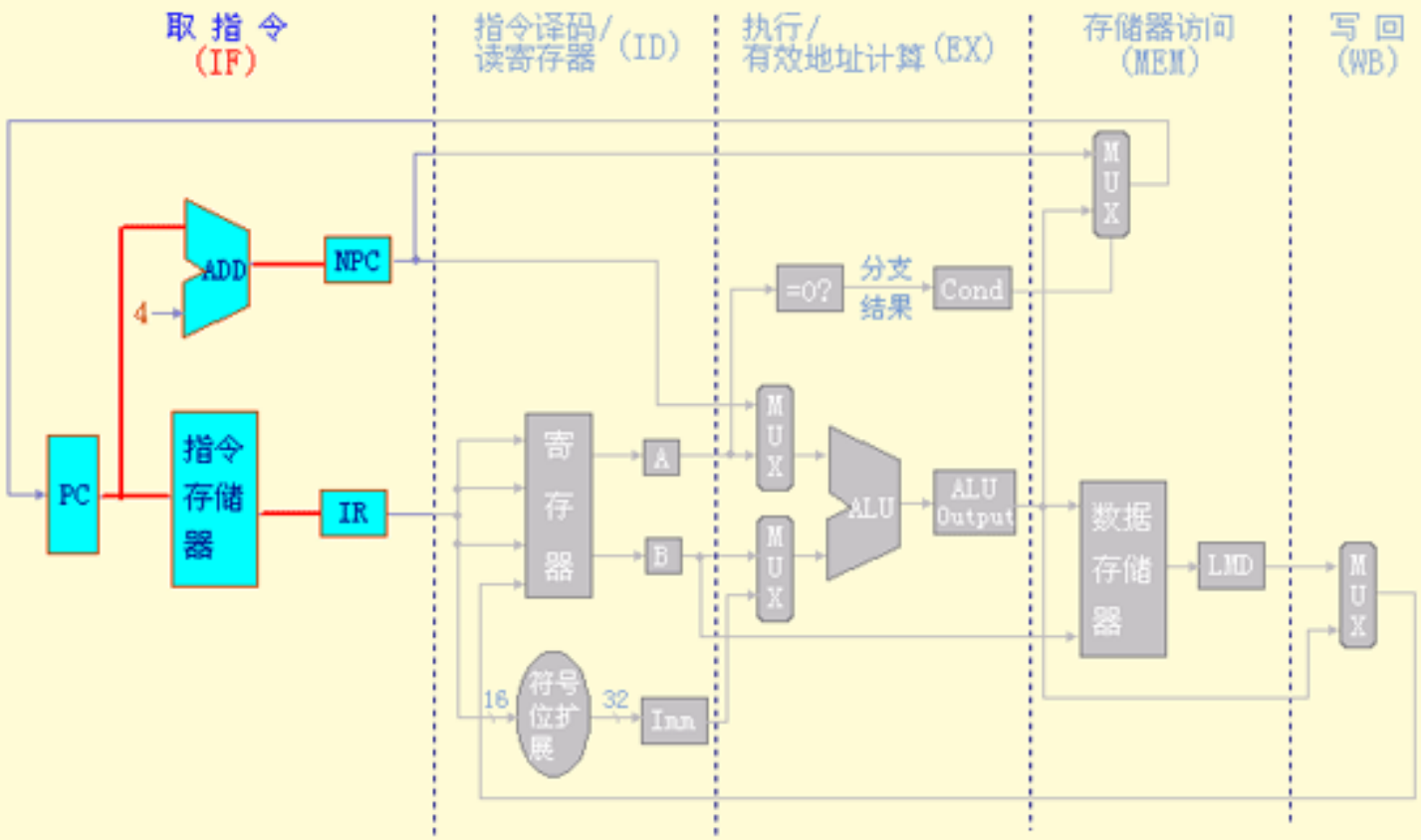
一般不采用这种方法，为什么？

- 流水技术

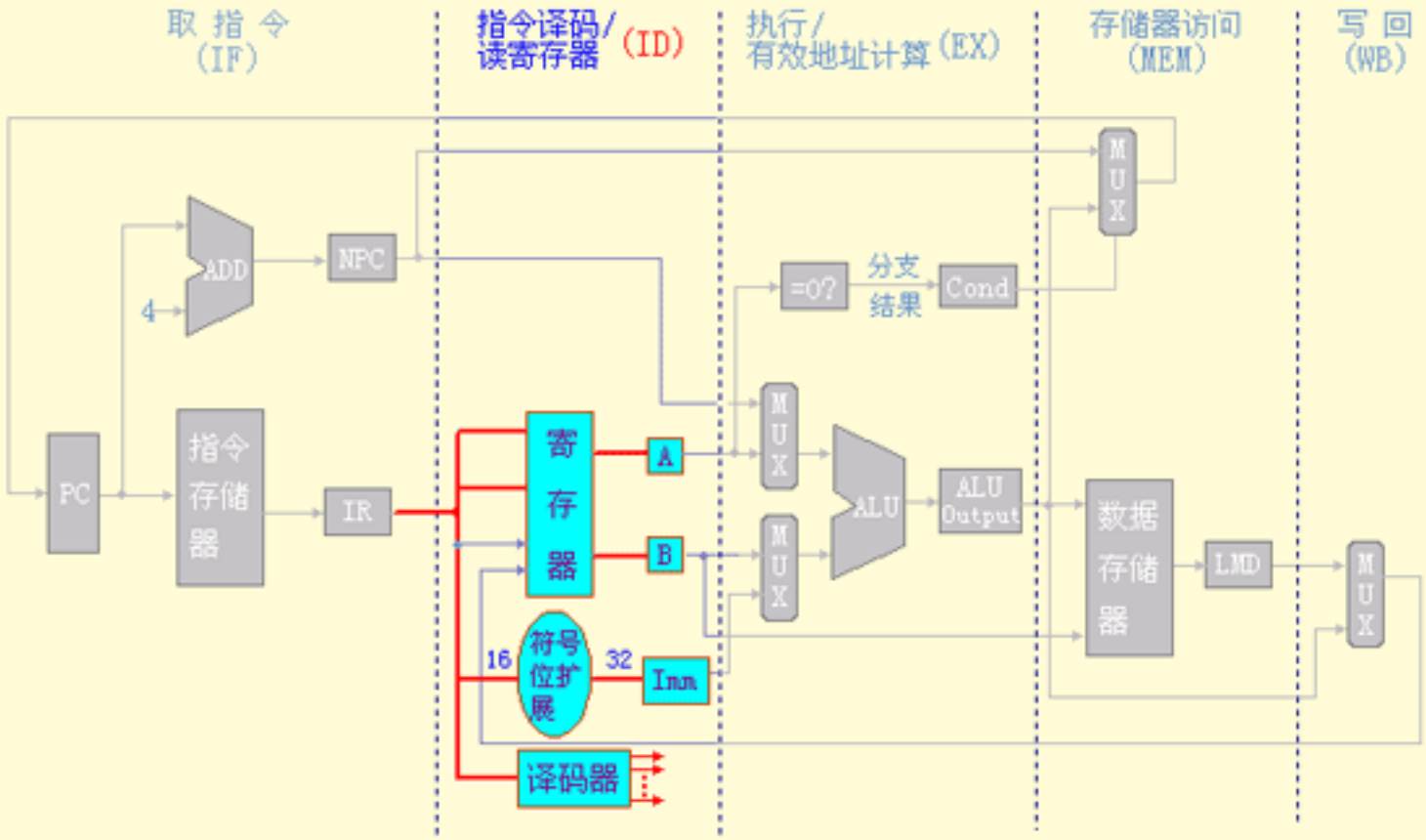
实现DLX指令的一种简单数据通路



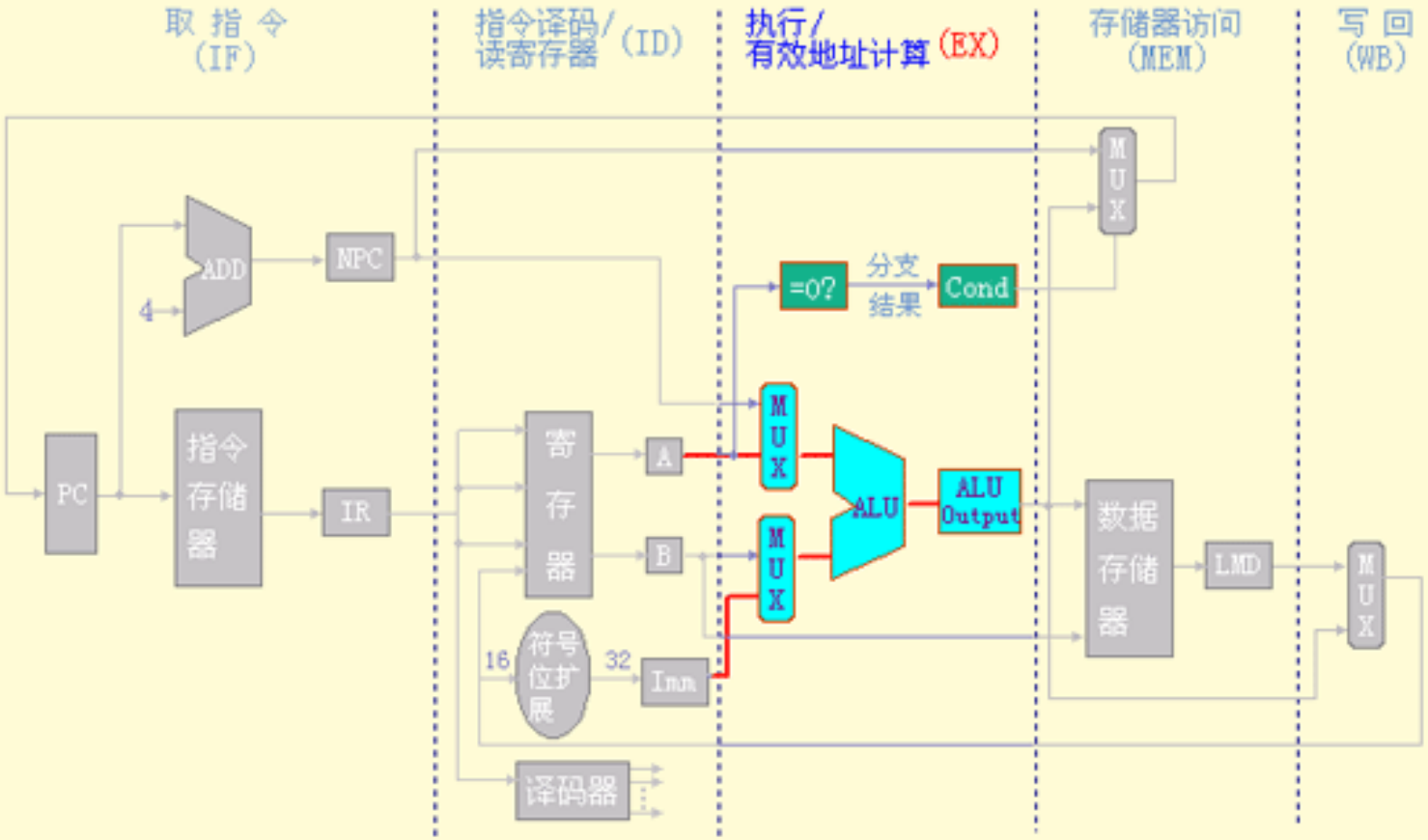
取指令周期的操作


取 指 令
(IF)
指令译码/
读寄存器 (ID)
执行/
有效地址计算 (EX)
存储器访问
(MEM)
写 回
(WB)
ADD
NPC
4
PC
指令
存储
器
IR
寄
存
器
A
B
符号
位扩
展
16
32
Imm
=0?
分支
结果
Cond
MUX
ALU
ALU
Output
数据
存储
器
LMD

指令译码/读寄存器周期的操作

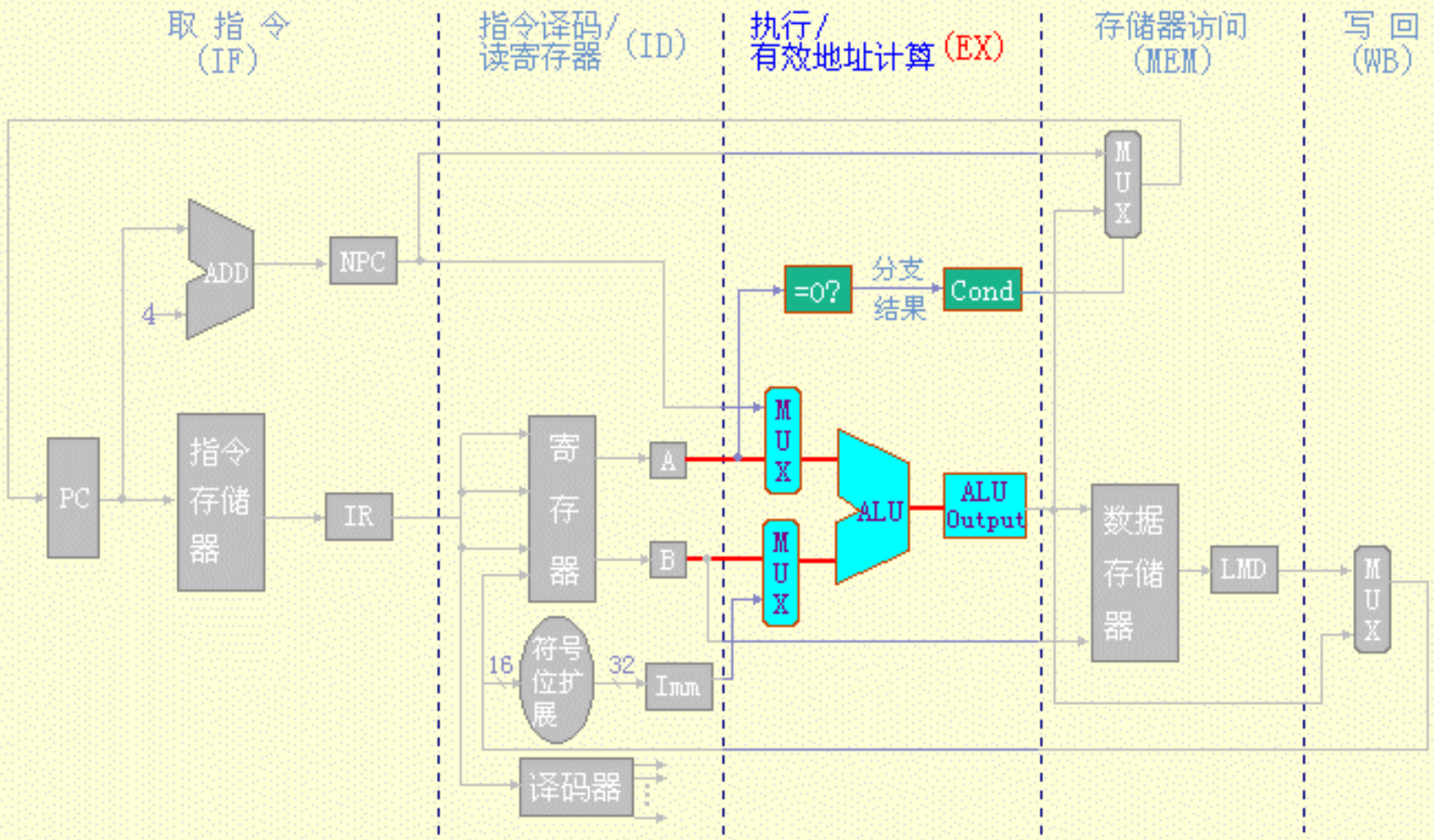


执行／有效地址计算周期的操作
（存储器访问指令）



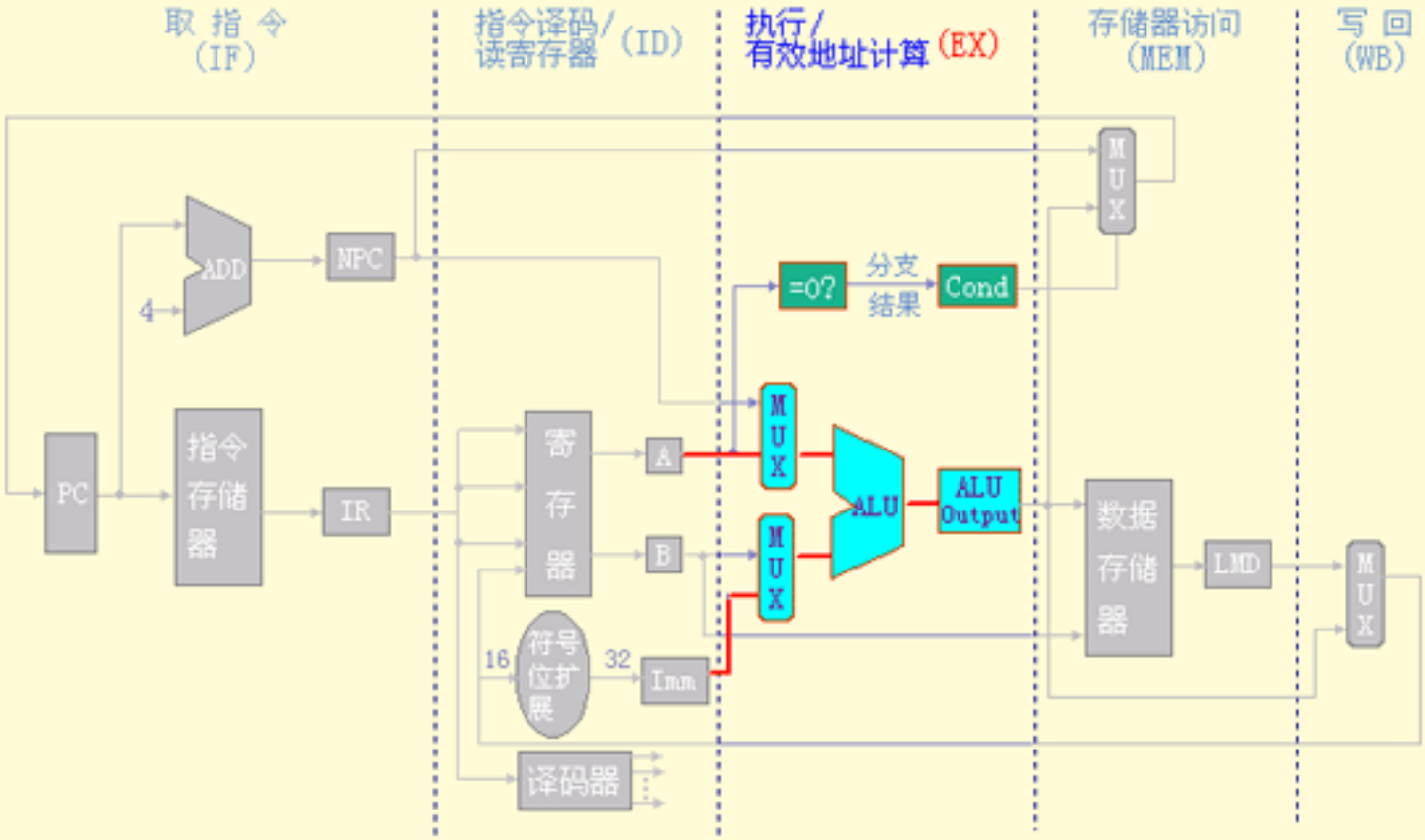
执行／有效地址计算周期的操作

（寄存器-寄存器ALU操作指令）

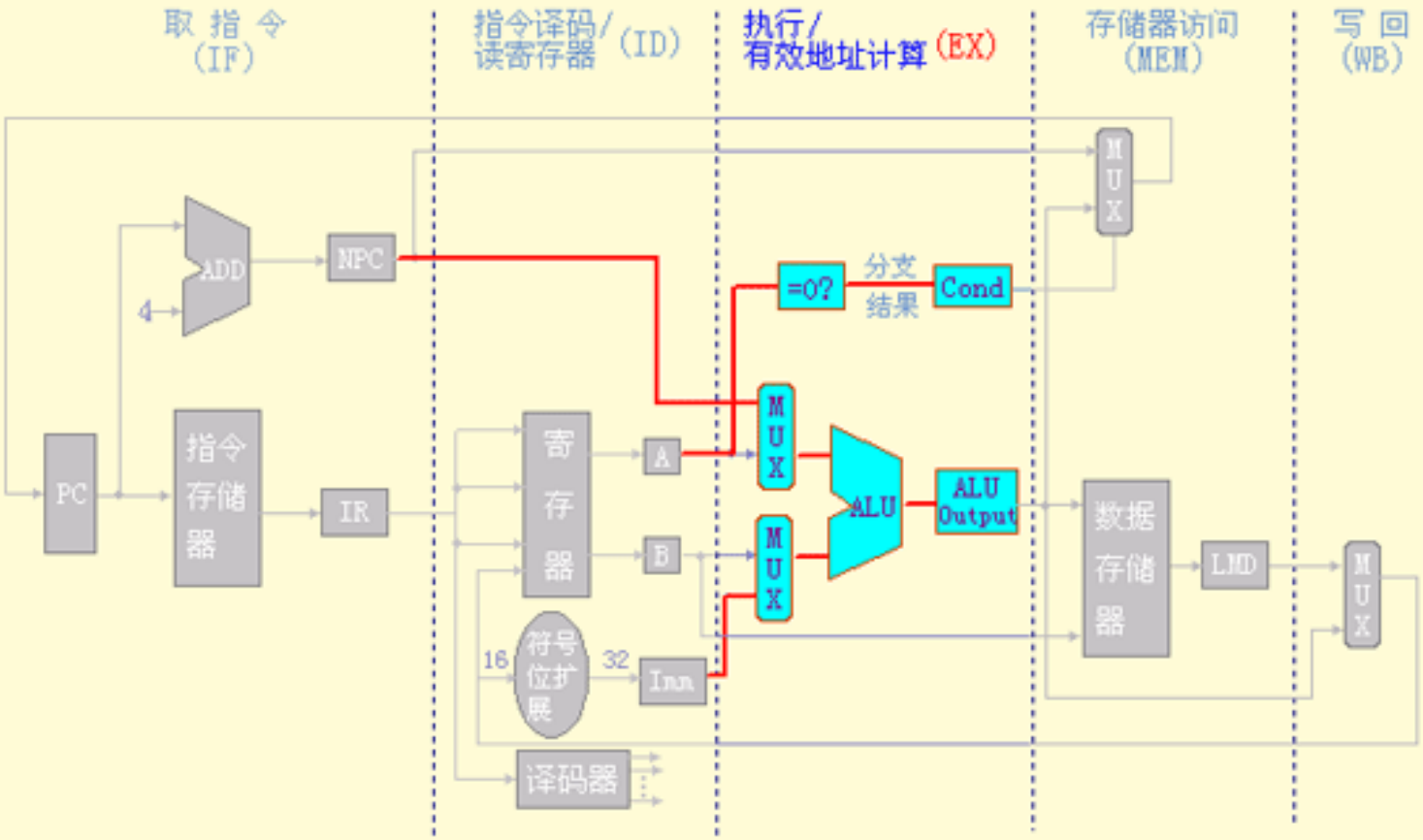


执行／有效地址计算周期的操作

（寄存器-立即值ALU操作指令）

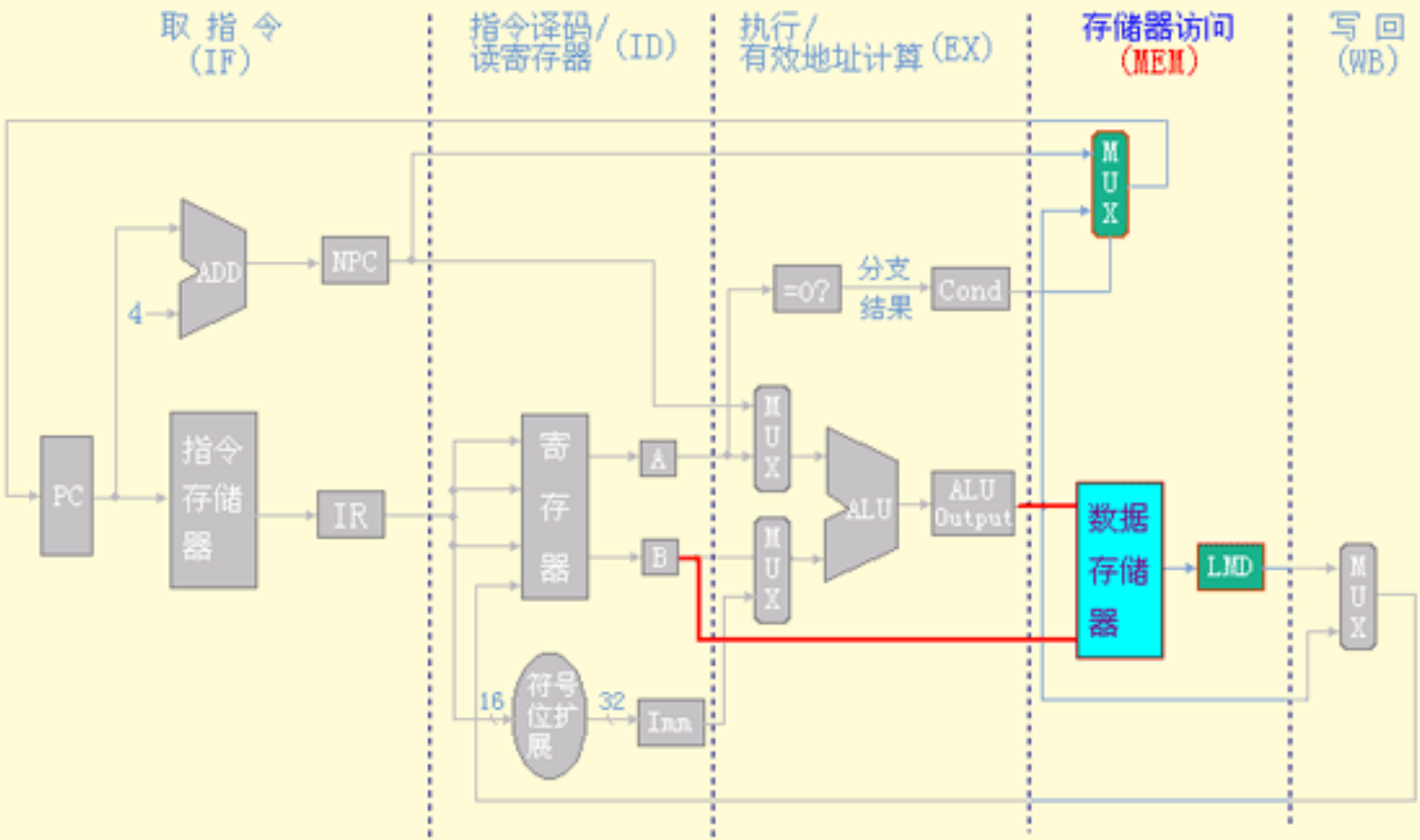


执行／有效地址计算周期的操作
（分支操作指令）

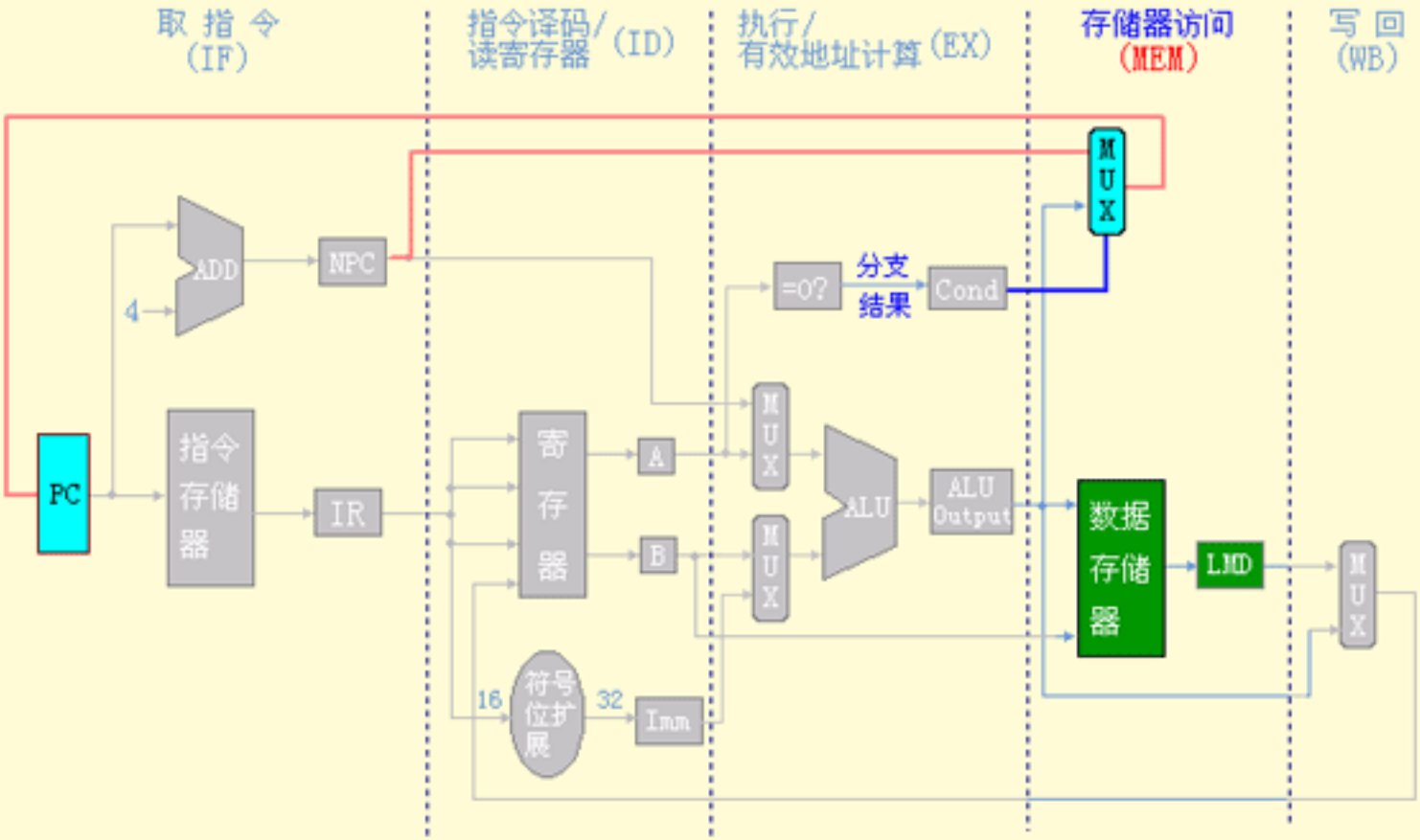


存储器访问/分支完成周期的操作

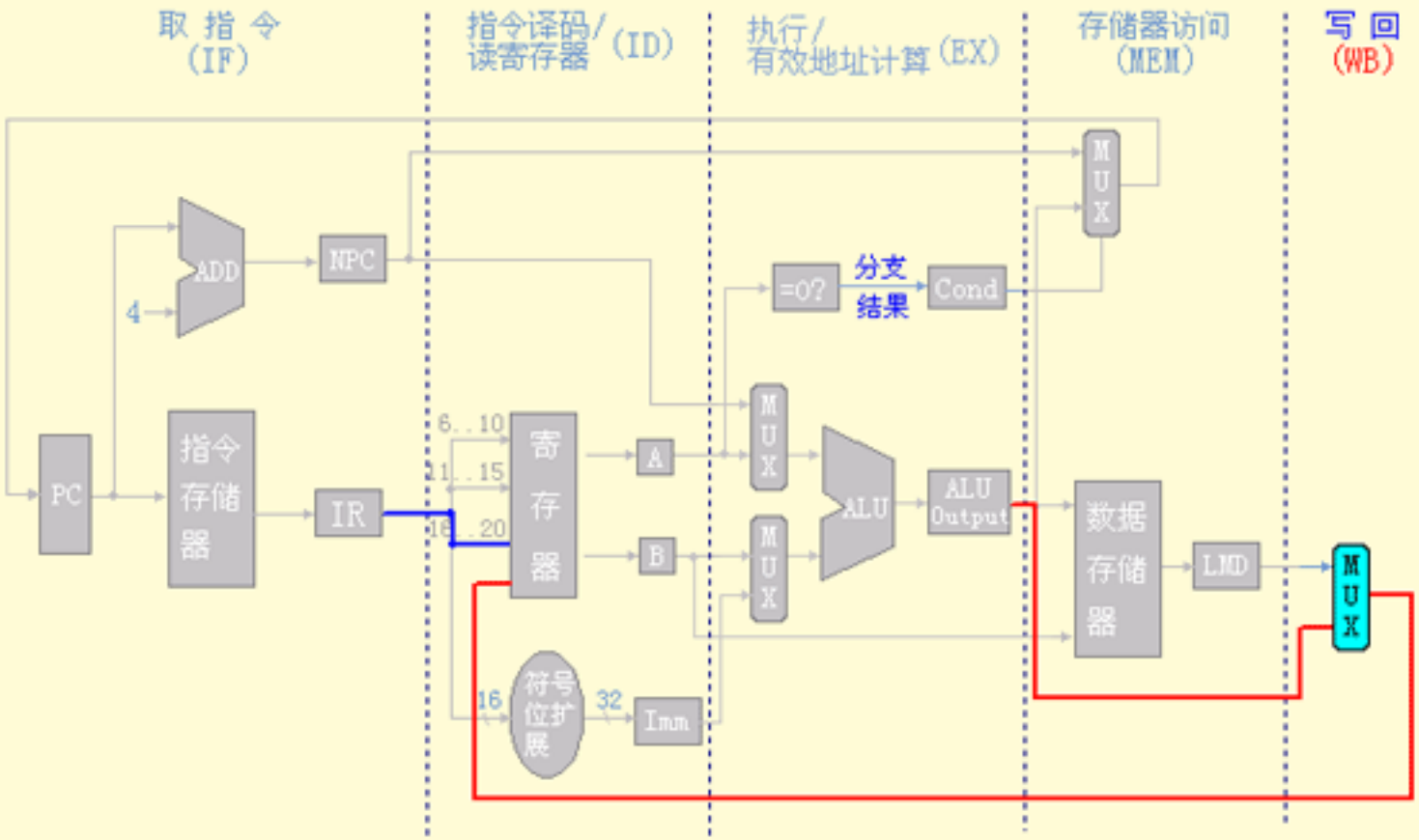
（存储器访问操作指令）



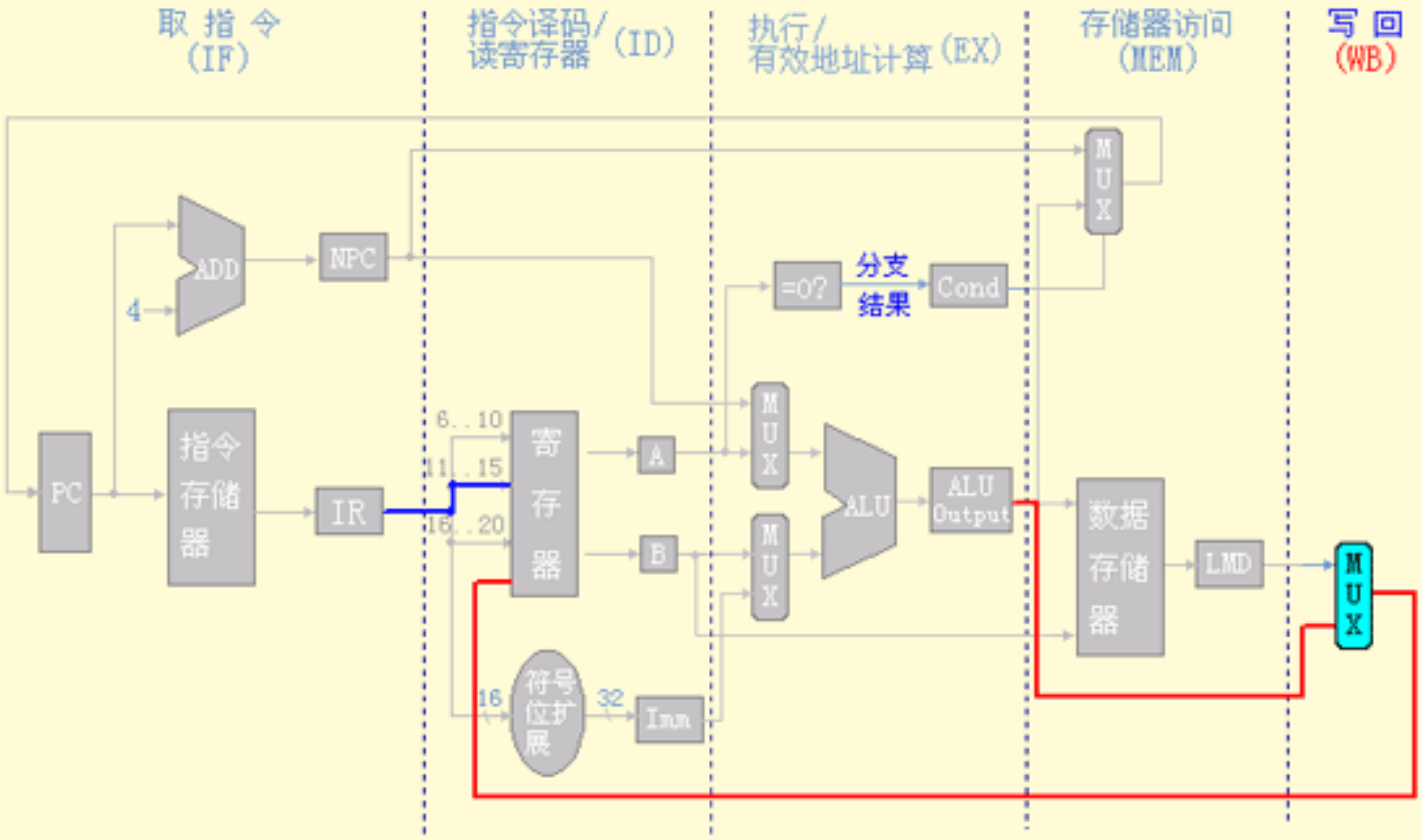
存储器访问/分支完成周期的操作
（分支操作指令）



写回周期的操作
（寄存器-寄存器型ALU指令）

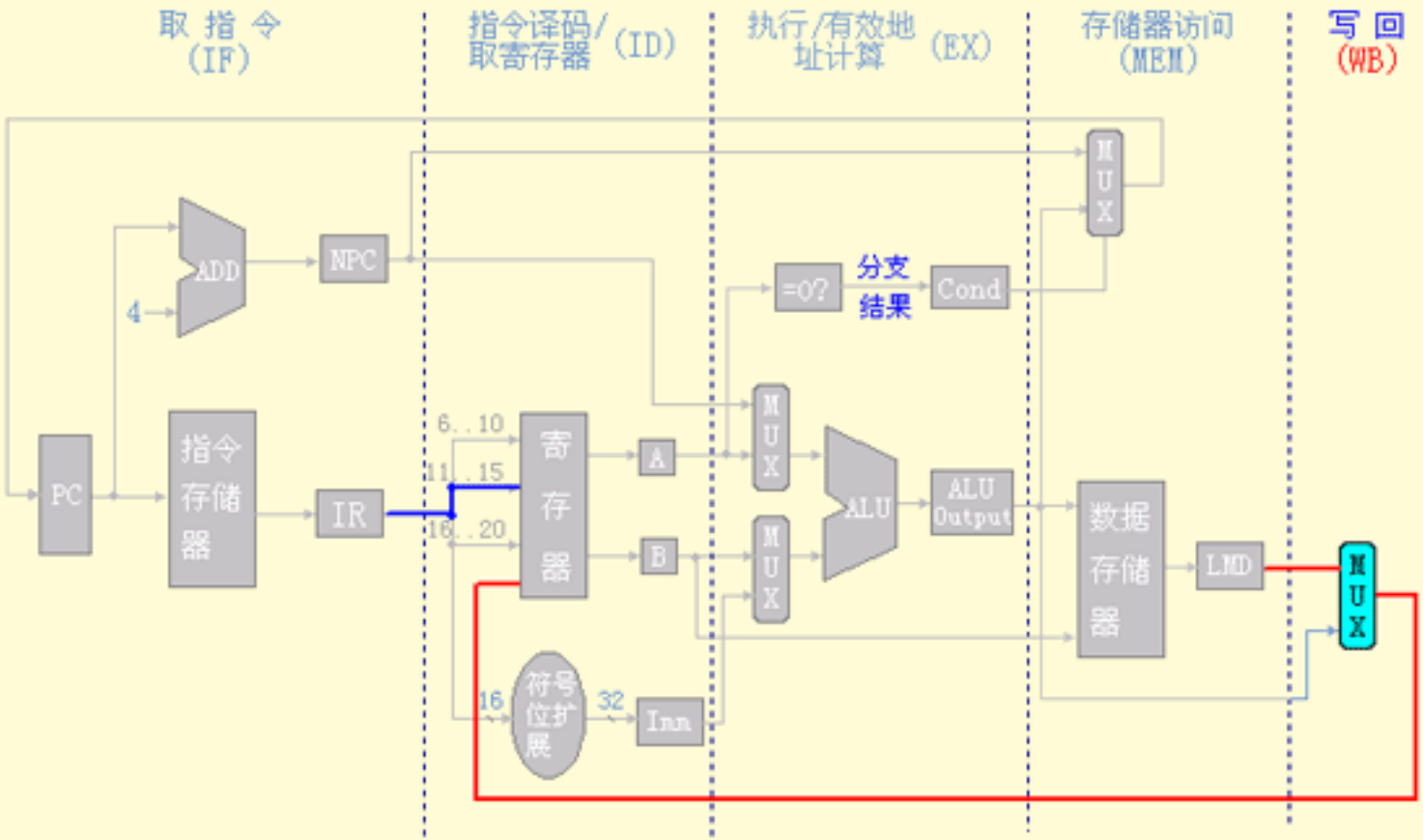


写回周期的操作
（寄存器-立即值型ALU指令）



写回周期的操作

（Load指令）



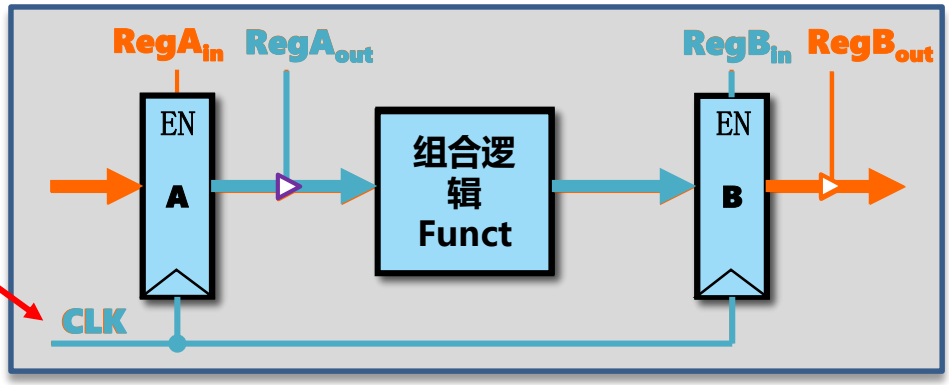
附录: CPU数据通路

数据通路 DataPath

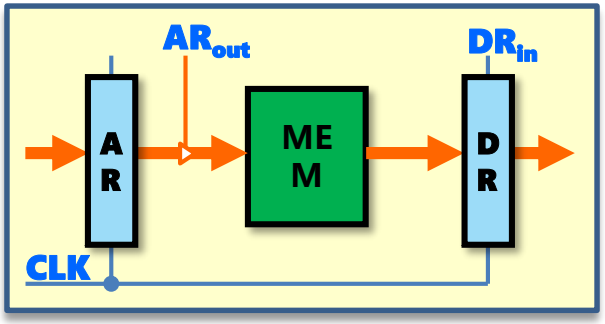
- 数据通路------执行部件间传送信息的路径 (数据流)
  - ◆通路的建立由控制信号控制，受时钟驱动 （控制流）
  - ◆不同指令、同一指令在执行的不同阶段的数据通路不同
  - ◆分类：共享通路（总线）、专用通路
    - 指令执行流程、执行效率
    - 微操作控制信号的时序安排

数据通路抽象模型（寄存器传输）

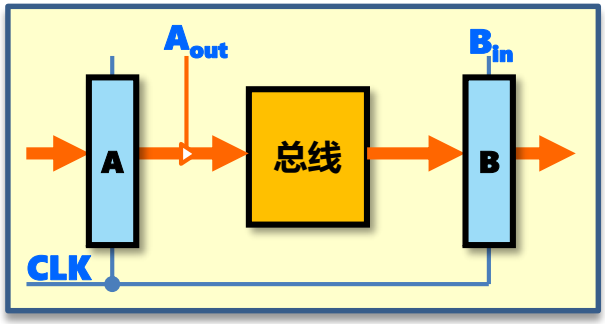
时钟频率?



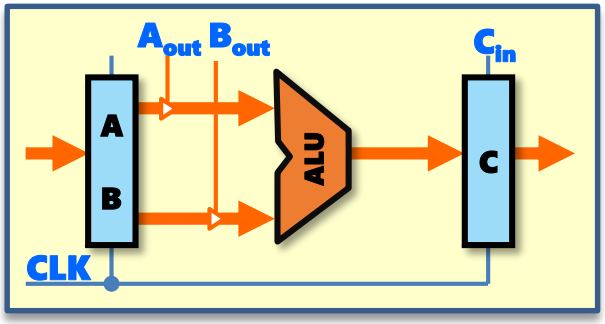
Funct (A)→ B



访存通路

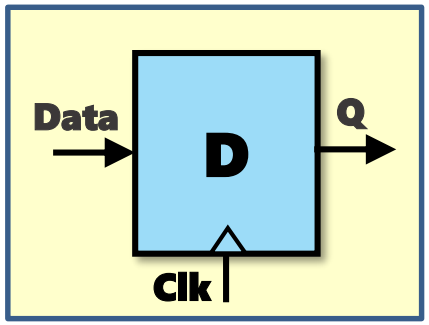


总线传输

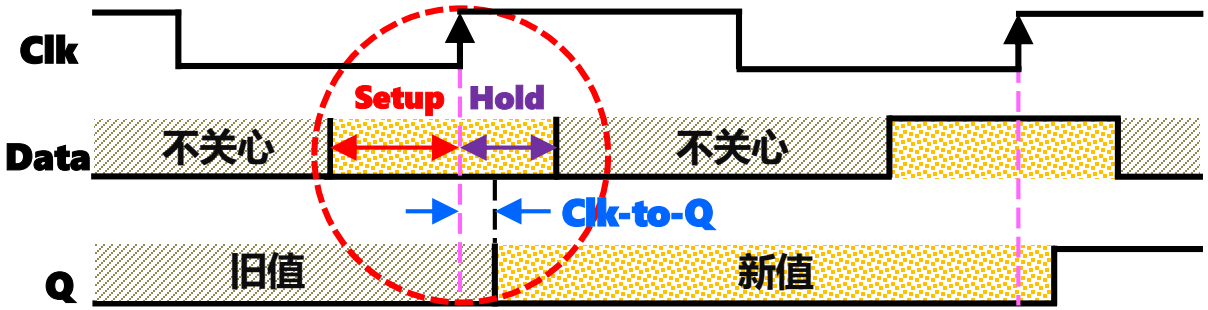


运算通路

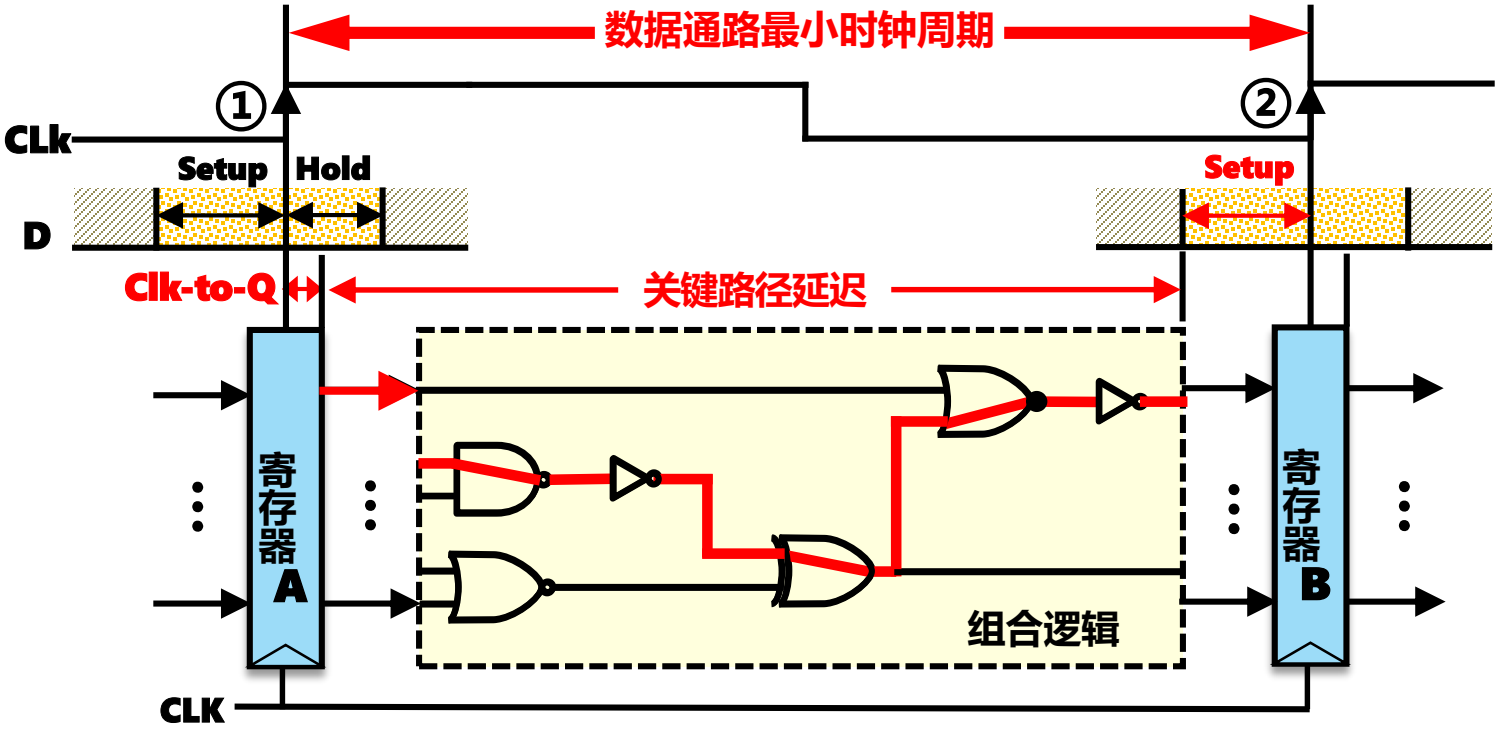
D触发器定时模型



- 时钟触发前输入须稳定一段 建立时间（Setup Time）
- 时钟触发后输入须稳定一段 保持时间（Hold Time）
- 时钟触发到输出稳定的时间 触发器延迟 Clk_to_Q



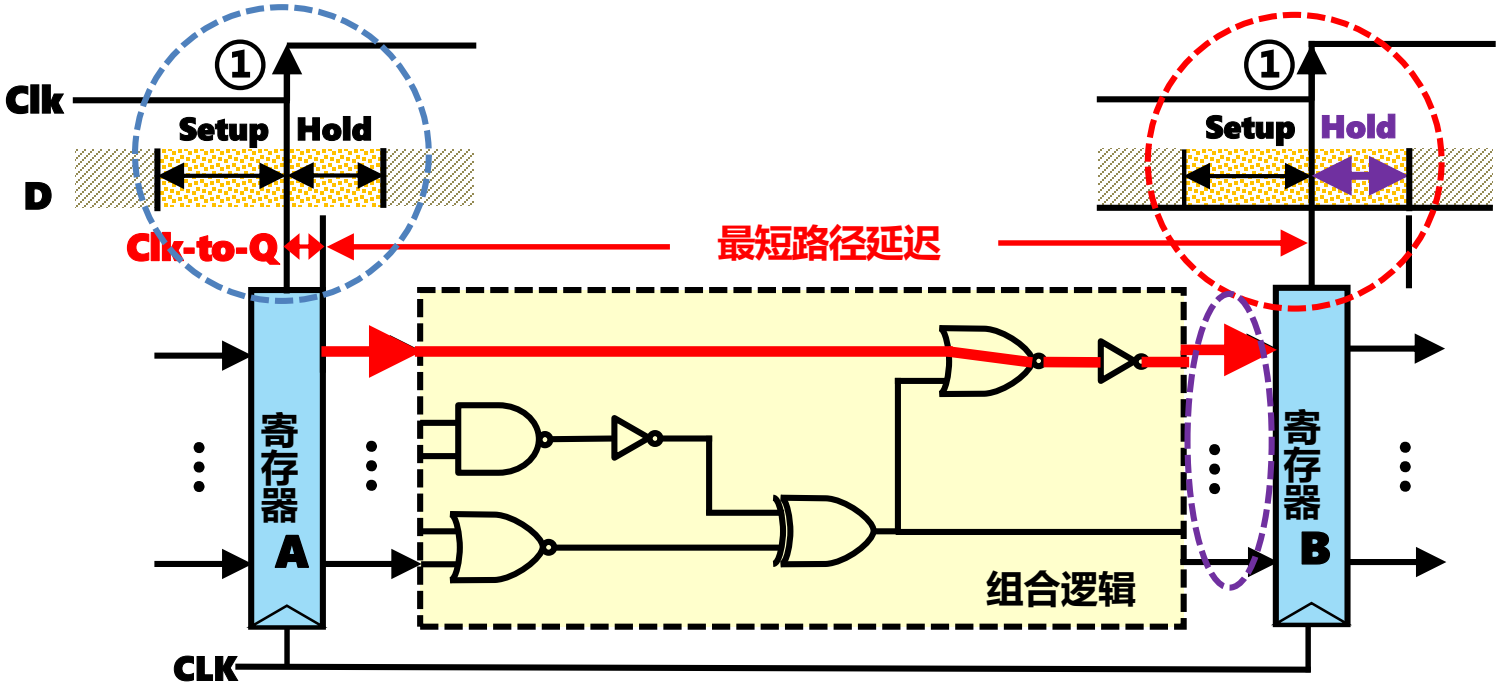
数据通路与时钟周期



- 时钟周期 > Clk_to_Q + 关键路径时延 + Setup Time

保持时间违例

- **Clk_to_Q** + 最短路径时延 > **Hold_Time**



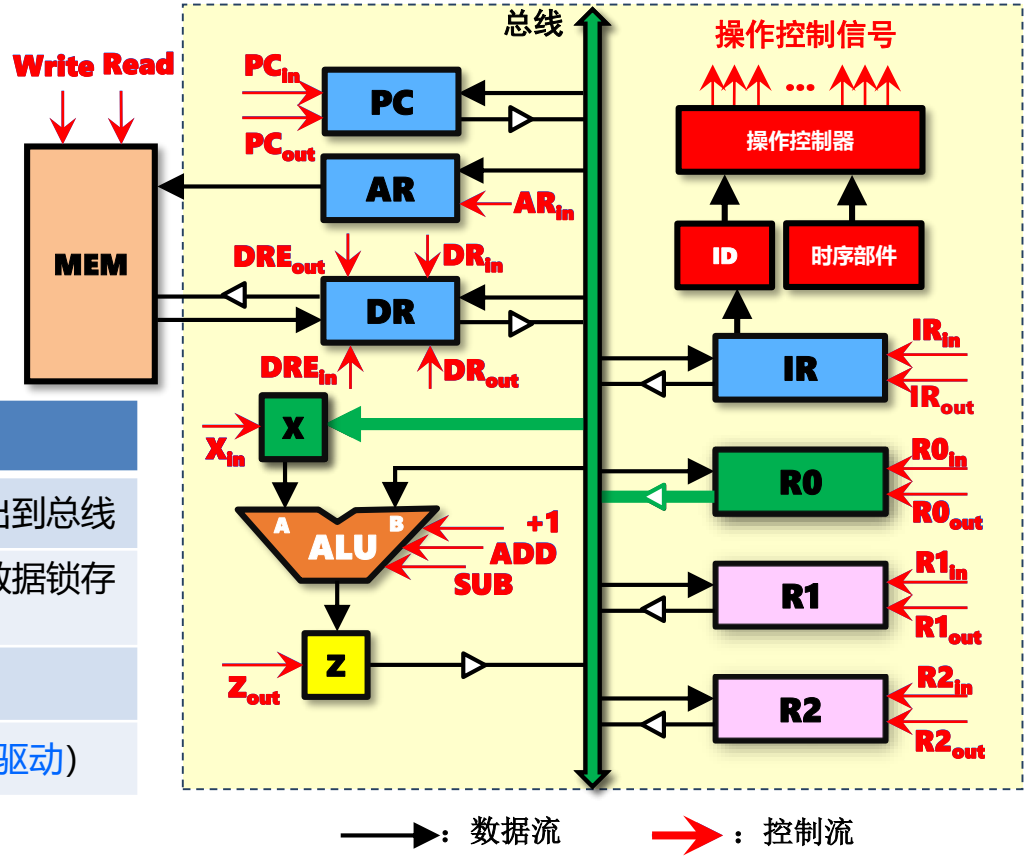
数据通路分类

- 共享通路（总线型）
  - ◆主要部件都连接在公共总线上，各部件间通过总线进行数据传输
  - ◆结构简单，实现容易，但并发性较差，需分时使用总线，效率低
- 专用通路
  - ◆并发度高，性能佳，设计复杂，成本高
  - ◆可以看作多总线结构

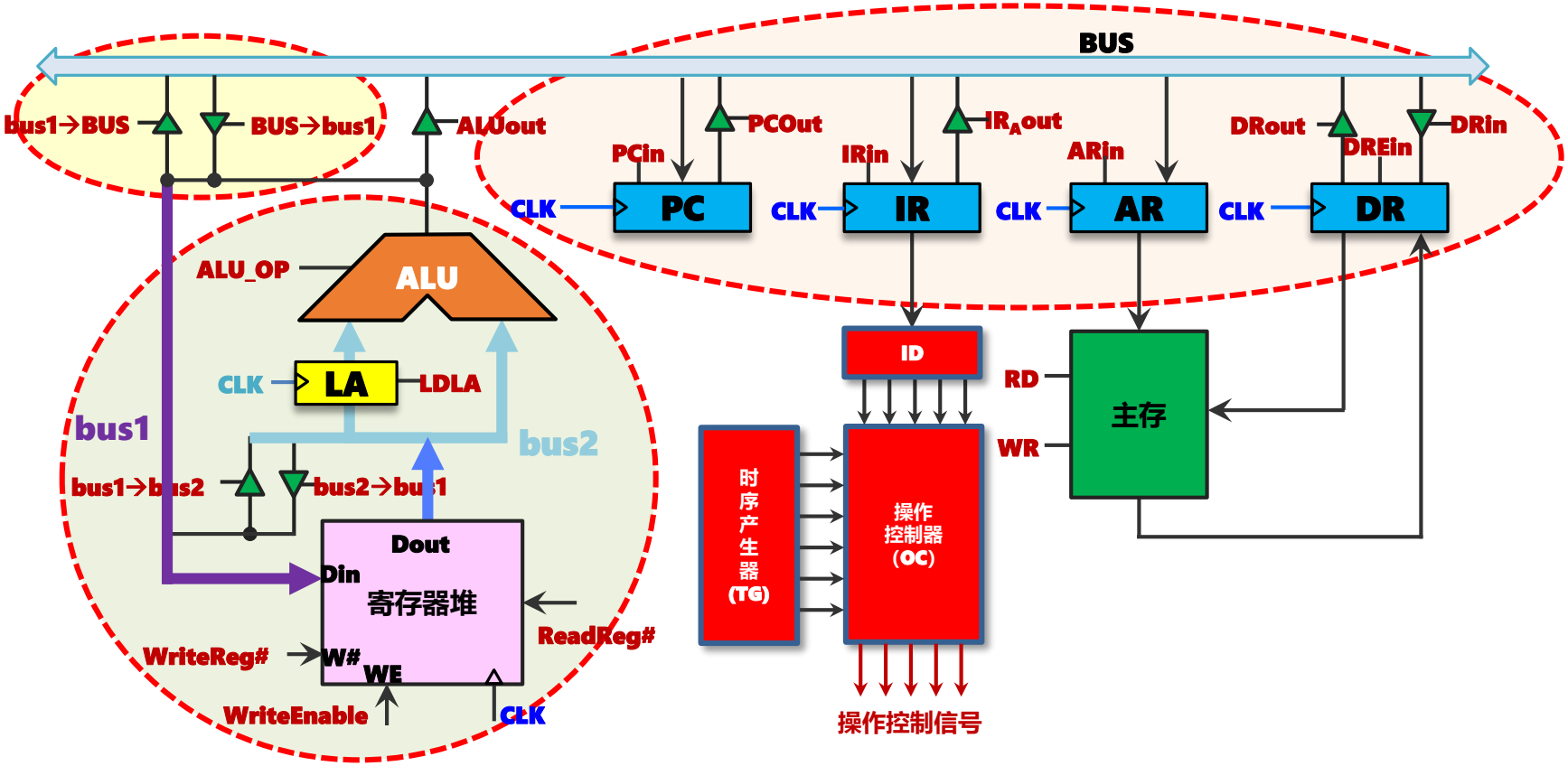
单总线结构CPU实例

- 主要部件都连接在总线上
- 各部件间通过总线进行传输

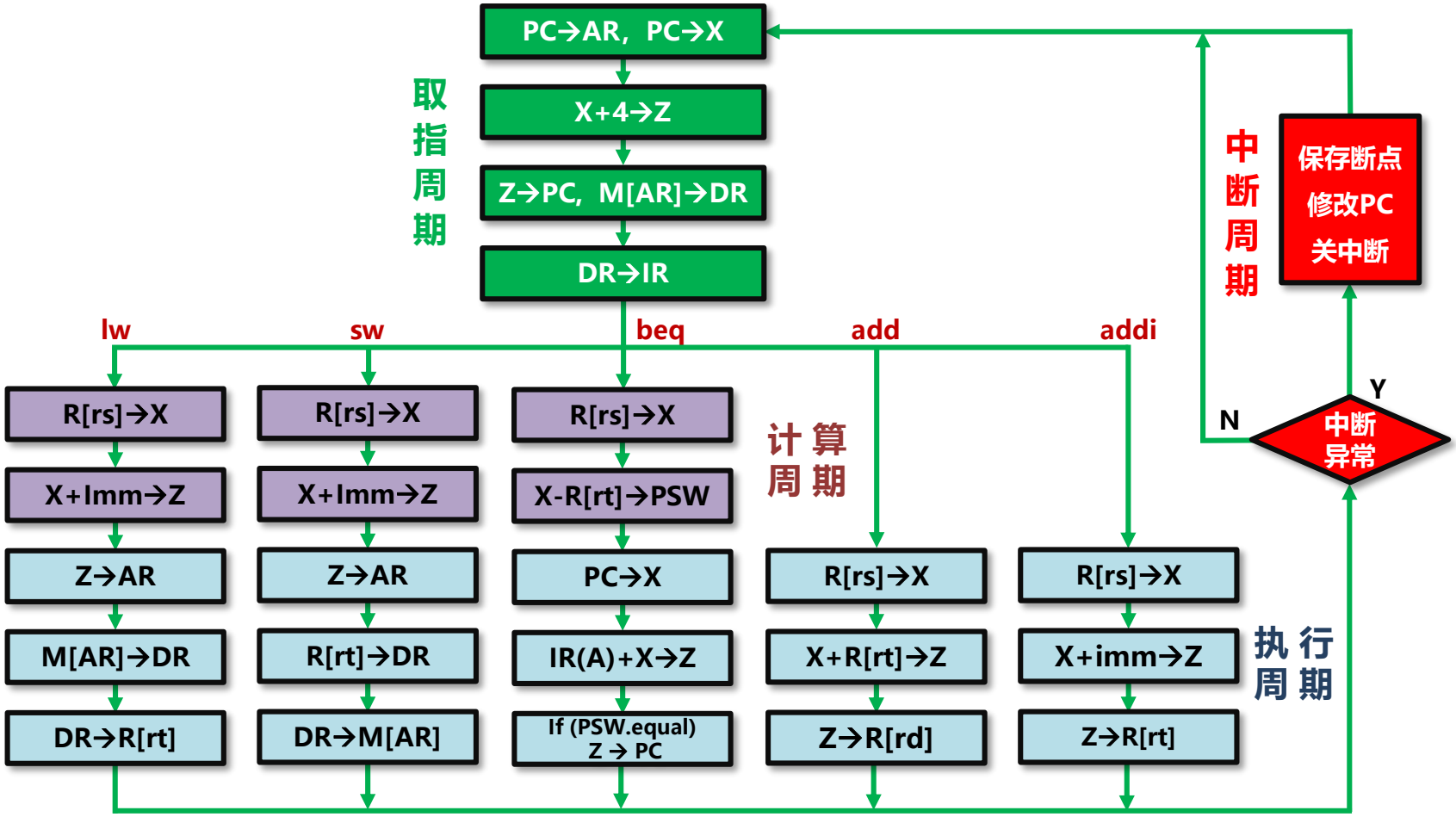
控制信号	作用说明
IR_{out}、PC_{out}、…$R1_{out}$	控制三态门将寄存器值输出到总线
IR_{in}、PC_{in}、…$R1_{in}$	控制寄存器使能端将总线数据锁存（时钟驱动）
+1、ADD、SUB	运算控制信号
Write、Read	内存读写控制信号（时钟驱动）



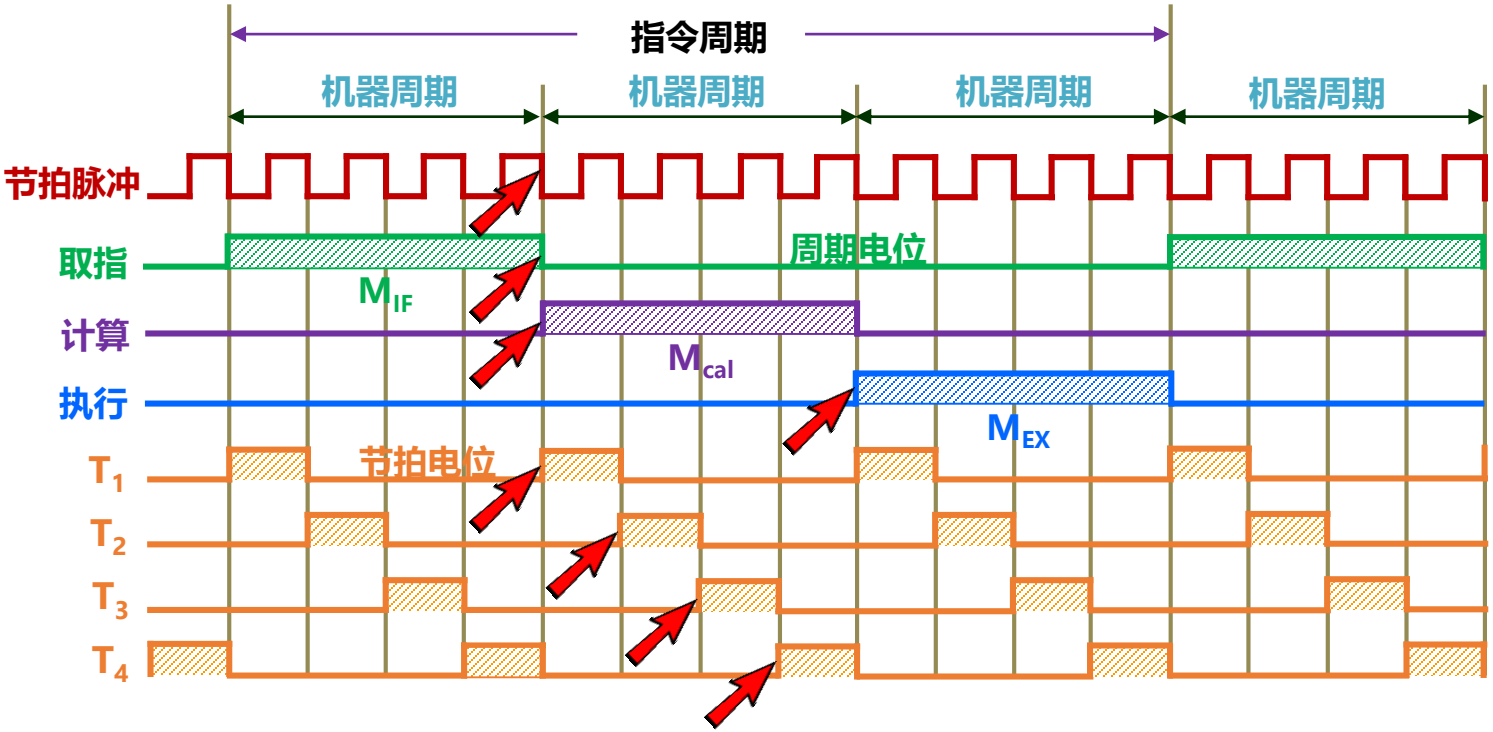
多总线架构数据通路


BUS
bus1→BUS
BUS→bus1
ALUout
PCin
PCOut
IRin
IRAout
ARin
DRout
DREin
DRin
CLK
PC
IR
AR
DR
ALU_OP
ALU
LA
LDLA
bus1
bus2
bus1→bus2
bus2→bus1
Dout
Din
寄存器堆
ReadReg#
WriteReg#
W#
WE
WriteEnable
ID
RD
WR
主存
时序产生器(TG)
操作控制器(OC)
操作控制信号

指令周期方框图 (数据流)

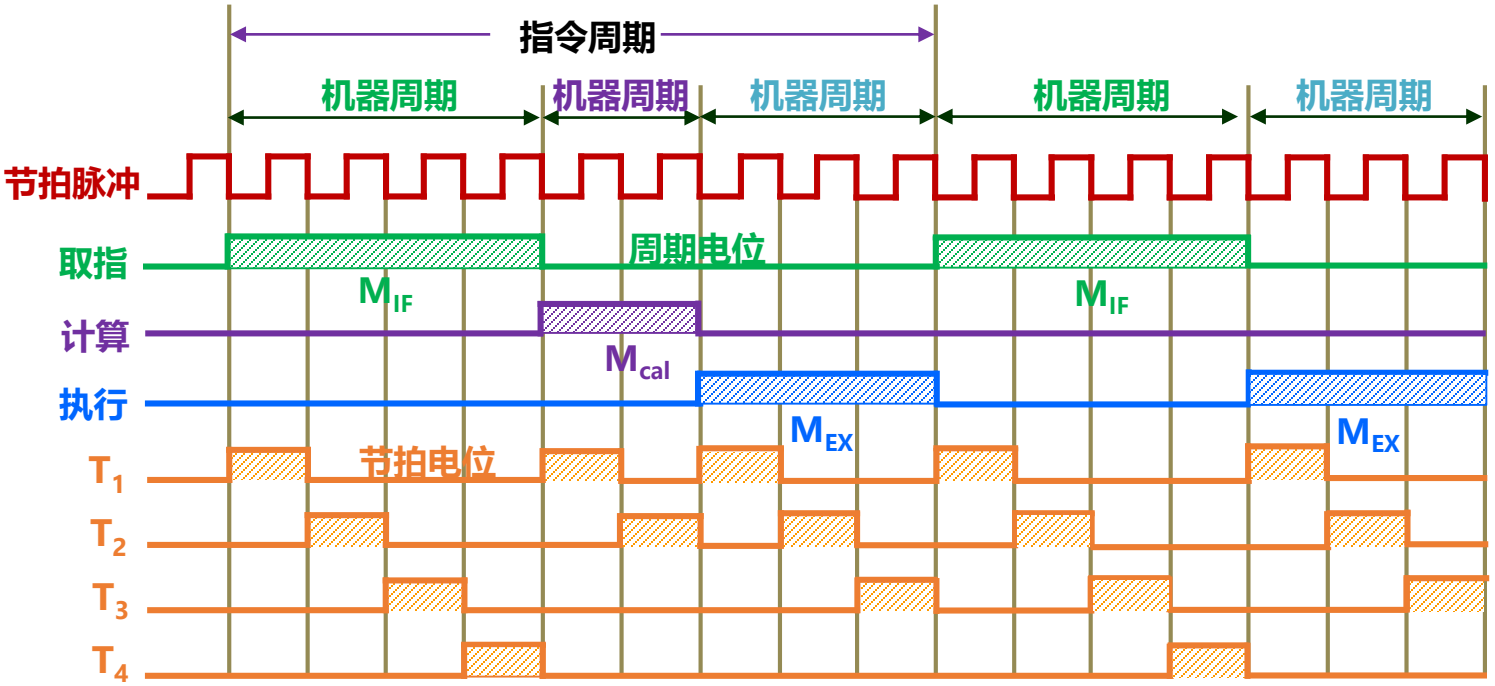

取指周期
PC→AR，PC→X
X+4→Z
Z→PC，M[AR]→DR
DR→IR
lw
sw
beq
add
addi
R[rs]→X
X+Imm→Z
Z→AR
M[AR]→DR
DR→R[rt]
R[rs]→X
X+Imm→Z
Z→AR
R[rt]→DR
DR→M[AR]
R[rs]→X
X-R[rt]→PSW
PC→X
IR(A)+X→Z
If (PSW.equal)
Z → PC
计算周期
R[rs]→X
X+R[rt]→Z
Z→R[rd]
R[rs]→X
X+imm→Z
Z→R[rt]
执行周期
中断周期
保存断点
修改PC
关中断
Y
N
中断异常

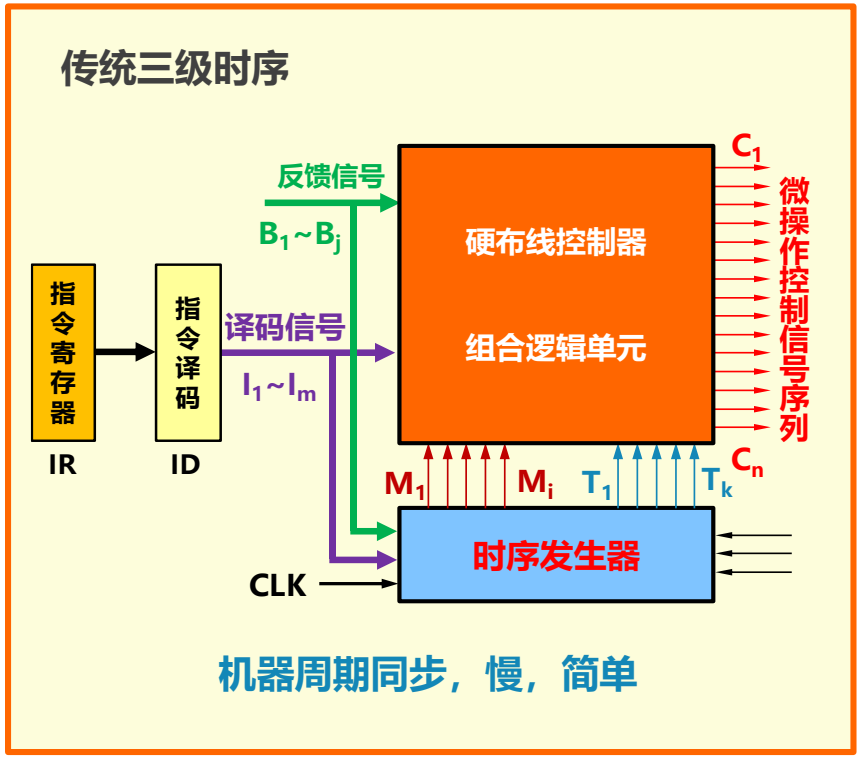
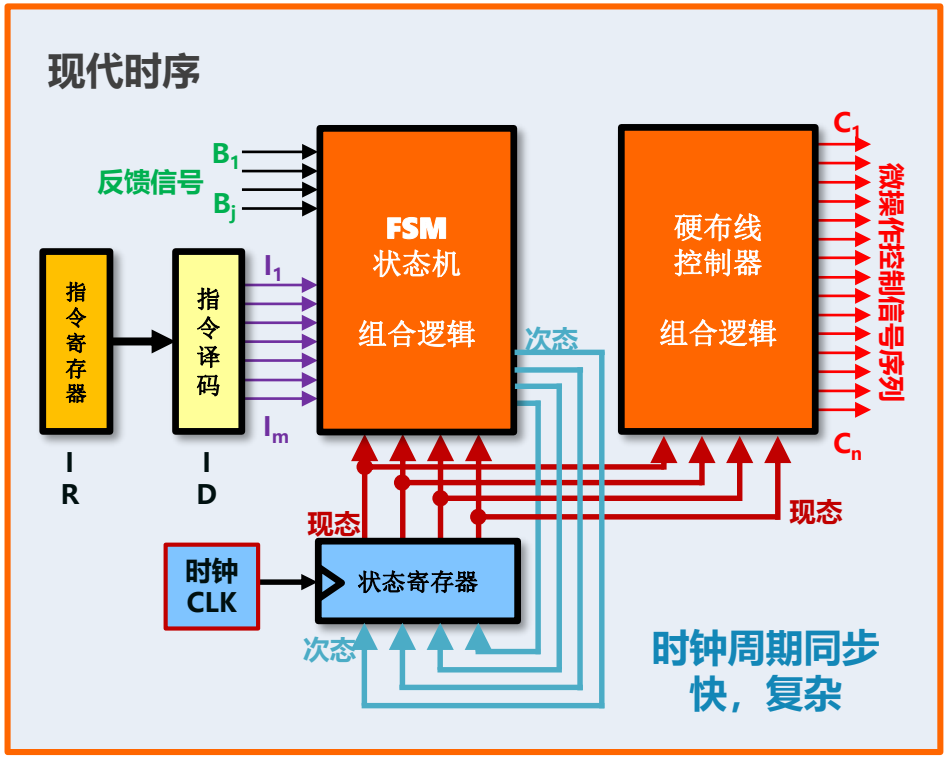
定长指令周期的三级时序发生器



构建时序发生器？ 输入：节拍脉冲 输出：M_{IF}，Mcal，M_{EX}，T1~T4

变长指令周期三级时序时序发生器



机器周期数可变、节拍数可变，无周期浪费，更加灵活

传统三级时序与现代时序对比